

Universidad Politécnica de Chiapas

Licenciatura en Ingeniería en Tecnologías de la Información e Innovación Digital

Cuatrimestre agosto - diciembre 2026

7° “C”

Materia: Optativa I: Base de datos en la nube

Examen práctico NoSQL Unidad II.
Paso a paso de la práctica

Alumno (a):

Mazariegos Laguna, Carlos Andrés. **243790**

Docente:

Ramos López, Montserrat Viridiana.

Suchiapa, Chiapas. 6 de noviembre de 2026.

Índice

Antes de empezar: qué vamos a hacer 3

Paso 1. Preparar la computadora 5

Paso 2. Crear el clúster gratuito en MongoDB Atlas 7

Paso 3. Autorizar mi IP (IP Access List) 8

Paso 4. Crear tres usuarios con permisos mínimos 9

Paso 5. Conectarme desde PowerShell 11

Paso 6. Cargar productos y pedidos (script 01) 13

Paso 7. Probar que las reglas funcionan (script 02) 14

Paso 8. Hacer consultas (script 03) 15

Paso 9. Cambios controlados (script 04) 17

Paso 10. Reportes y velocidad con índice (script 05) 18

Paso 11. Crear la base de grafos en Neo4j AuraDB 20

Paso 12. Cargar el grafo de servicios 21

Paso 13. Ver el grafo y contarlo 24

Paso 14. ¿Qué se afecta si falla Pagos? 26

Paso 15. Probar los permisos del analista 28

Paso 16. Probar los permisos del operador 29

Paso 17. Monitoreo: qué significan las gráficas 30

Paso 18. Sacar un respaldo con mongodump 32

Paso 19. Restaurar el respaldo en otra base 33

Paso 20. Comprobar que la copia es idéntica 34

Antes de empezar: qué vamos a hacer

En esta práctica armo la base de datos de una tienda ficticia llamada **Mercado Ágil**. La tienda tiene productos, recibe pedidos y funciona con varios servicios (pagos, inventario, notificaciones…) que dependen unos de otros. Para eso uso dos bases de datos NoSQL gratuitas en la nube:

- **MongoDB Atlas**: guarda los productos y los pedidos como **documentos** (parecidos a fichas con datos).
- **Neo4j AuraDB**: guarda los servicios como un **grafo** (círculos unidos con flechas) para ver qué depende de qué.

La práctica sigue el orden de la rúbrica: primero el modelo de datos, luego las reglas de integridad, después las consultas y cambios, el grafo, la administración (usuarios, monitoreo y respaldo) y al final la verificación.

En este documento los comandos van en **negritas** y con letra de máquina de escribir, así: **`cls`**. Cuando un comando tiene una comilla invertida (`) al final de la línea, significa que continúa en el siguiente renglón; se copian todas las líneas juntas.

Palabras que voy a usar mucho

Palabra	Qué es, en sencillo
Clúster	El servidor que Atlas me presta en la nube. En el plan gratuito M0 son 3 computadoras que guardan la misma información.
Base de datos	Un contenedor con nombre. Aquí se llama comercio_u2.
Colección	Un grupo de documentos del mismo tipo. Aquí hay dos: productos y pedidos.
Documento	Un registro, escrito en formato parecido a JSON: { clave: valor }. Cada pedido es un documento.
mongosh	El programa de terminal con el que le escribo comandos a MongoDB.
Script (.js)	Un archivo con muchos comandos guardados. En vez de escribirlos uno por uno, los ejecuto todos con load().
Nodo y relación	En Neo4j, un nodo es una cosa (un servicio) y una relación es la flecha que los une (DEPENDE_DE).
Cypher	El lenguaje para hacer consultas en Neo4j.

Tres reglas de seguridad que seguí en toda la práctica

1. Nunca mostrar contraseñas en capturas, documentos ni chats. Las guardé en una nota privada.
2. Usar solo planes gratuitos (Free / M0 / AuraDB Free), sin tarjeta.
3. Dar a cada usuario solo los permisos que necesita (principio de mínimo privilegio).

Paso 1. Preparar la computadora

Todo se hizo en Windows con **PowerShell** (Menú Inicio, escribir PowerShell y Enter). Primero instalé dos herramientas gratuitas de MongoDB: **mongosh**, para escribir comandos, y **Database Tools**, que trae mongodump y mongorestore para el respaldo.

```
winget install MongoDB.Shell
winget install MongoDB.DatabaseTools
```

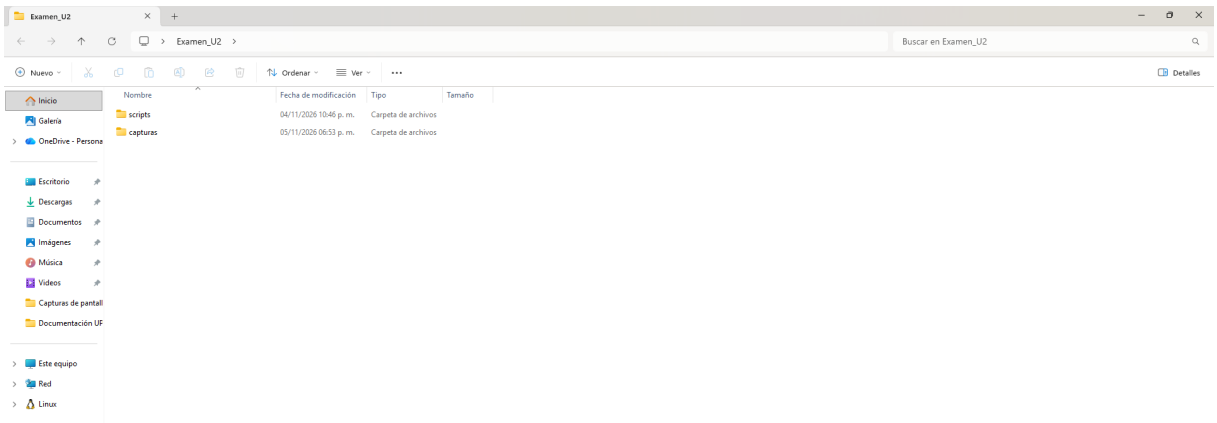
Después cerré y volví a abrir PowerShell, porque Windows solo reconoce los programas nuevos en una ventana nueva. Para comprobar que quedaron instalados:

```
mongosh --version
mongodump --version
```

¿Qué significa? si sale un número de versión, el programa está listo. Si sale «no se reconoce como nombre de un cmdlet», hay que cerrar y abrir PowerShell; si sigue igual, falta agregar la carpeta de Database Tools a la variable PATH de Windows.

La carpeta de trabajo

Toda la práctica vive en una carpeta llamada **Examen_U2** en el Escritorio. Su ruta completa es **C:\Users\carlo\OneDrive\Desktop\Examen_U2**. Adentro tiene la carpeta **scripts** (los 10 archivos de comandos) y la carpeta **capturas** (las evidencias).



Cada vez que abro PowerShell, lo primero es entrar a esa carpeta, porque los scripts se buscan desde ahí:

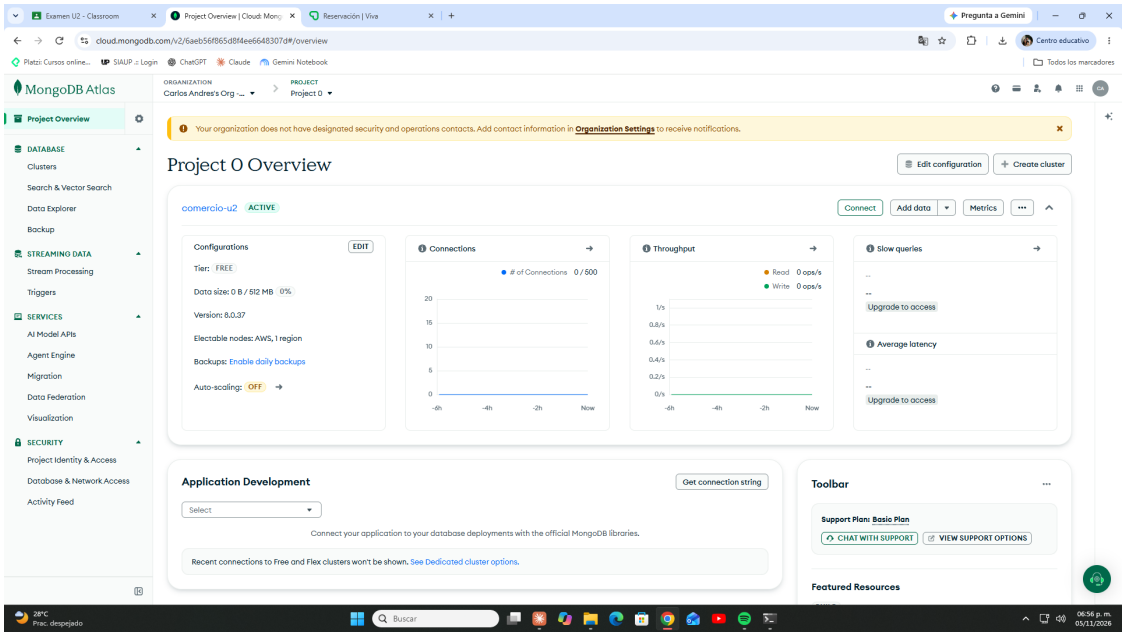
```
cd C:\Users\carlo\OneDrive\Desktop\Examen_U2
```

Archivo	Para qué sirve
01_productos_pedidos.js	Crea los 5 productos, el validador y los 12 pedidos.

Archivo	Para qué sirve
02_pruebas_integridad.js	Intenta meter datos malos para comprobar que se rechazan.
03_consultas.js	Seis búsquedas de pedidos.
04_cambios_controlados.js	Actualiza y borra datos con condiciones de seguridad.
05_agregaciones_indice.js	Reportes con totales y prueba de velocidad con índice.
06a / 06b	Pruebas de permisos del analista y del operador.
07_verificar_restauracion.js	Compara la base original con la restaurada.
servicios.cypher	Crea el grafo de servicios en Neo4j.
respaldo_comandos.txt	Los comandos de respaldo y restauración.

Paso 2. Crear el clúster gratuito en MongoDB Atlas

1. Entré a **`cloud.mongodb.com`** e inicié sesión.
2. Dentro de **Project 0** creé un clúster llamado **comercio-u2** con el plan **M0 / Free** (costo cero).
3. Esperé a que dijera **ACTIVE**.
4. Importante: desactivé el traductor de Chrome en esta página, porque traduce los nombres técnicos y a veces hace que la página falle.



Cómo leer esta pantalla

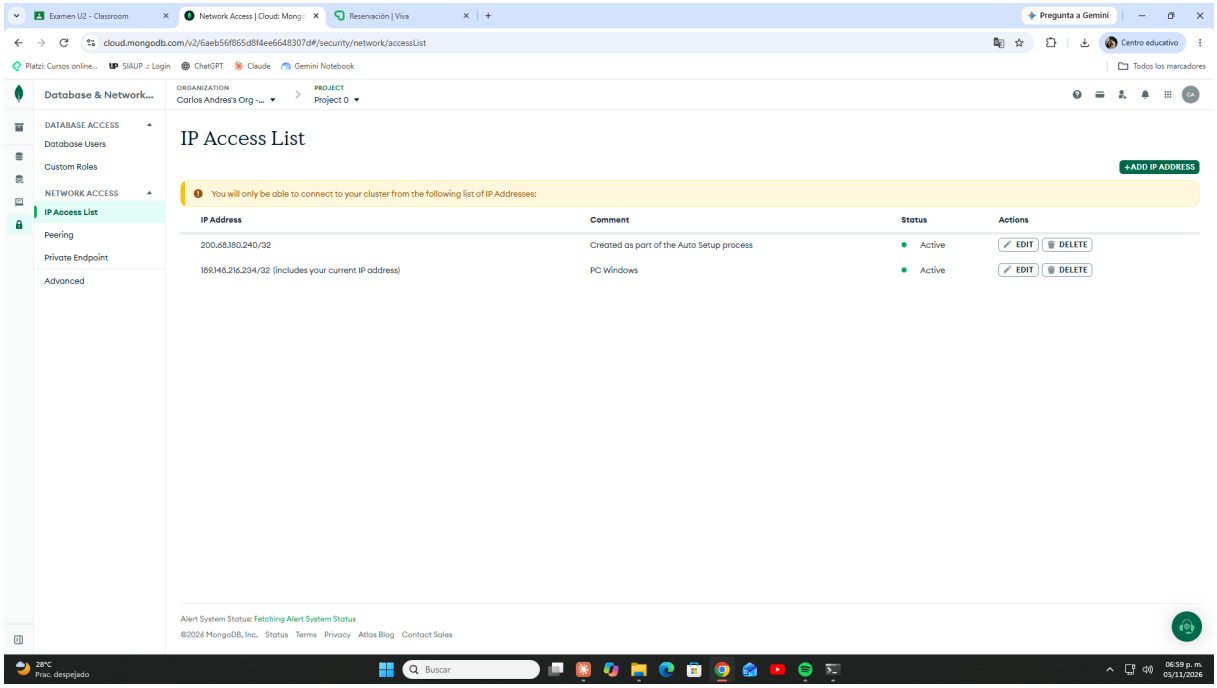
Parte	Qué me dice
ACTIVE	El clúster está encendido y listo para recibir conexiones.
Tier: FREE	Es el plan gratuito M0. La rúbrica pide comprobarlo.
Data size: 0 B / 512 MB	Cuánto espacio uso de los 512 MB gratis. Sale 0 porque aún no cargaba datos.
Version 8.0.37	La versión de MongoDB que corre en el clúster.
Connections 0 / 500	Cuántas conexiones hay abiertas en ese momento, de un máximo de 500 en el plan gratuito.
Throughput (Read / Write)	Cuántas lecturas y escrituras por segundo recibe la base. En 0 porque nadie la estaba usando.
Slow queries	Consultas lentas. En el plan gratuito pide «Upgrade to access» porque es una función de pago.

Cómo lo explico: *«Creé un clúster M0 gratuito en Atlas. Aquí se ve que está activo, que es Free y que todavía no tiene datos.»*

Paso 3. Autorizar mi IP (IP Access List)

Atlas no deja entrar a cualquiera. Primero hay que decirle **desde qué computadoras** se permite conectarse. Eso se hace con la dirección IP.

1. En el menú: **Database & Network Access → IP Access List**.
2. Clic en **+ ADD IP ADDRESS**.
3. Clic en el botón **ADD CURRENT IP ADDRESS** (pone sola la IP de mi computadora).
4. En el comentario escribí **PC Windows** y di clic en **Confirm**.
5. Esperé a que el estado dijera **Active**.

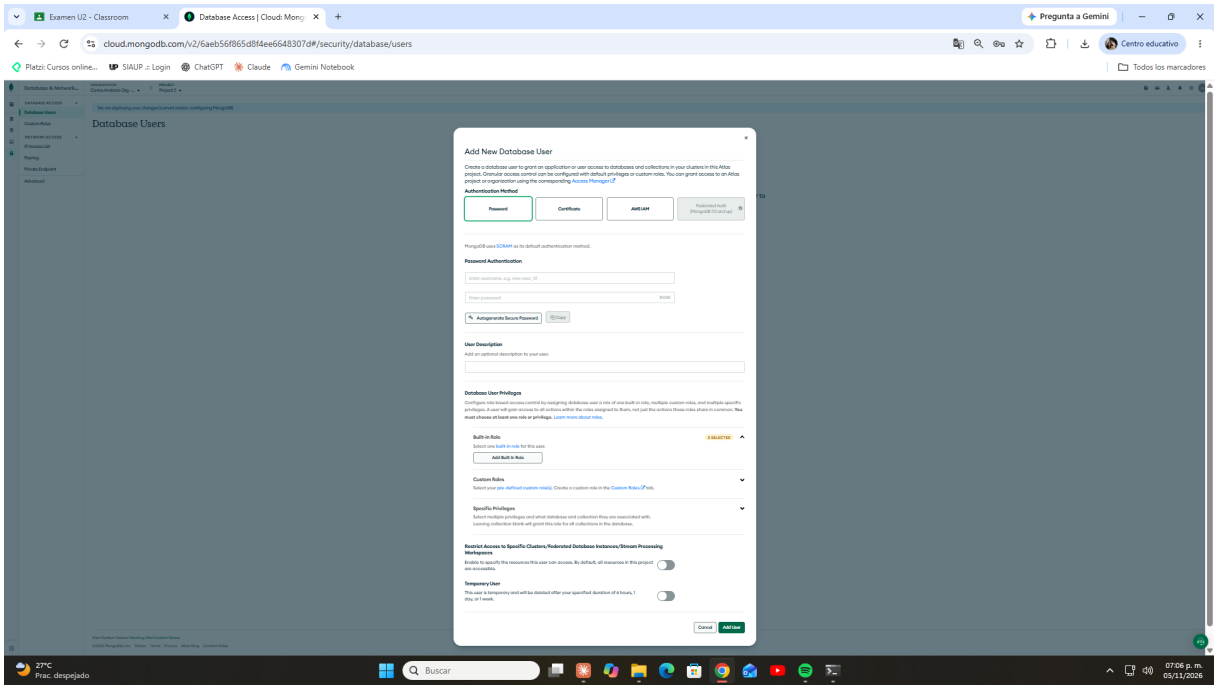


¿Qué significa? la terminación ***/32*** significa «exactamente esta IP, ninguna otra». La segunda dice «includes your current IP address», o sea que es la computadora que estoy usando. Si mi IP cambia (por ejemplo, otra red de internet), tengo que agregar la nueva.

Cómo lo explico: *«La lista de IP es el primer candado: decide desde dónde se puede llegar al clúster, pero no decide qué puede hacer cada usuario.»*

Paso 4. Crear tres usuarios con permisos mínimos

Cada persona que usa la base tiene un trabajo distinto, así que cada una recibe solo los permisos que necesita. Esto se hace en **Database & Network Access → Database Users → + ADD NEW DATABASE USER**.

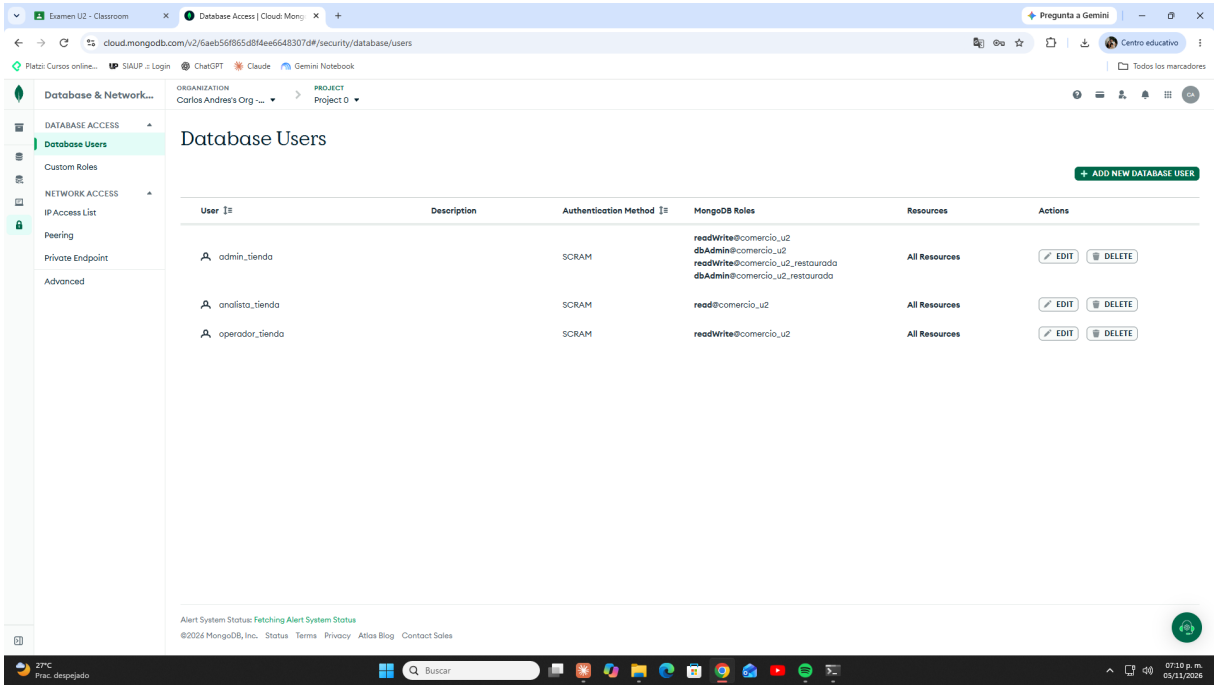


En esa ventana llené, para cada usuario:

1. **Authentication Method:** Password.
2. **Username:** el nombre del usuario.
3. **Password:** clic en **Autogenerate Secure Password** y luego en **Copy**; la pegué en mi nota privada.
4. **Built-in Role** y **Custom Roles:** los dejé vacíos.
5. **Specific Privileges:** agregué los permisos de la tabla de abajo, dejando vacía la colección (así aplica a toda la base).
6. Clic en **Add User** y esperé a que desapareciera el aviso «We are deploying your changes».

Usuario	Permisos (rol @ base)	Por qué
admin_tienda	readWrite y dbAdmin en comercio_u2 y en comercio_u2_restaurada	Carga datos, crea el validador y los índices, y hace respaldo y restauración.
analista_tienda	read en comercio_u2	Solo consulta reportes; no debe cambiar nada.
operador_tienda	readWrite en comercio_u2	Registra y actualiza pedidos, pero solo en la base de la tienda.

Al final la lista quedó así:



¿Qué significa? read = solo leer. **readWrite** = leer, insertar, actualizar y borrar documentos. **dbAdmin** = administrar la estructura (índices, validadores, estadísticas), sin poder crear usuarios. **SCRAM** es el método de contraseña segura que usa MongoDB. Ningún usuario tiene «Atlas admin», que sería darle control total del clúster.

Cómo lo explico: *«Apliqué el principio de mínimo privilegio: si alguien robara la contraseña del analista, solo podría leer, no modificar ni borrar.»*

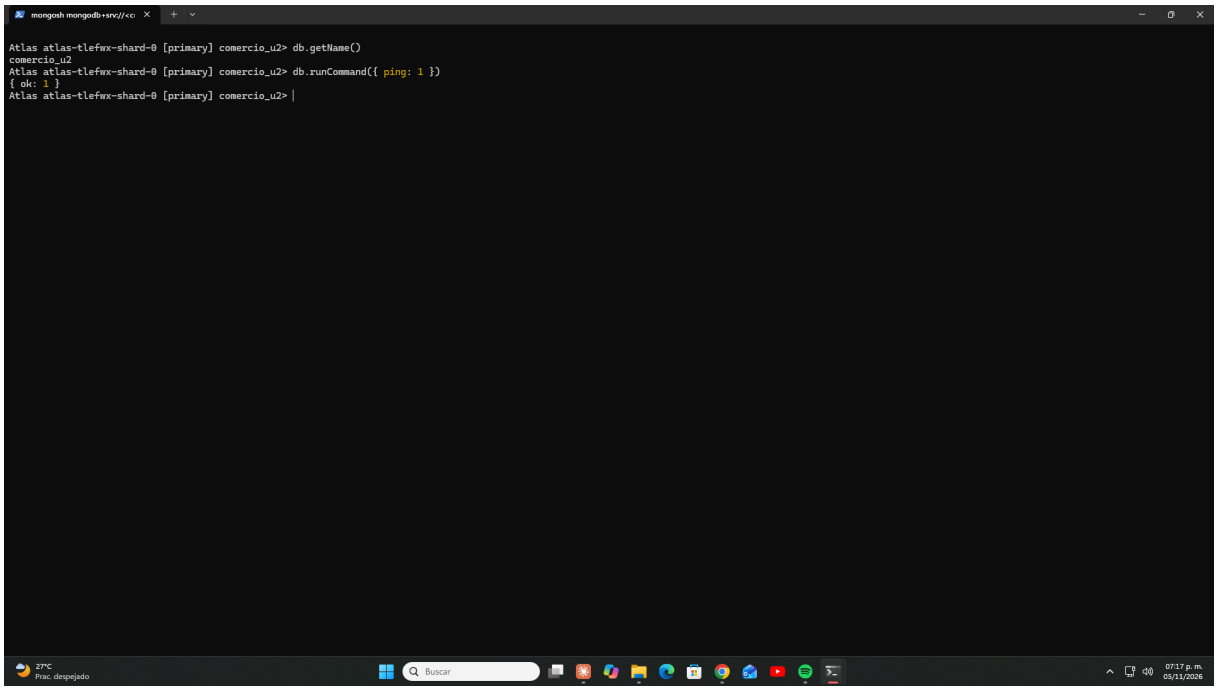
Paso 5. Conectarme desde PowerShell

Para conectarme necesito el **host**, que es la dirección de mi clúster en internet. Lo saqué de **Clusters → Connect → Shell**. Atlas muestra un comando y de ahí copié solo la parte **comercio-u2.zepcsbi.mongodb.net**. El host no es secreto; la contraseña sí.

```
cd C:\Users\carlo\OneDrive\Desktop\Examen_U2
mongosh "mongodb+srv://comercio-u2.zepcsbi.mongodb.net/comercio_u2" `
  --apiVersion 1 --username admin_tienda
```

Cuando pide **Enter password**, pegué la contraseña con **clic derecho** y Enter. Al pegarla **no se ve nada** en la pantalla, ni asteriscos; es normal. Ya conectado, comprobé:

```
cls
db.getName()
db.runCommand({ ping: 1 })
```



¿Qué significa? `cls` limpia la pantalla. **`db.getName()`** dice en qué base estoy: **comercio_u2**. **`ping`** le pregunta al servidor si está vivo, y **{ ok: 1 }** significa «sí, aquí estoy». El texto **[primary]** indica que estoy conectado al servidor principal del clúster.

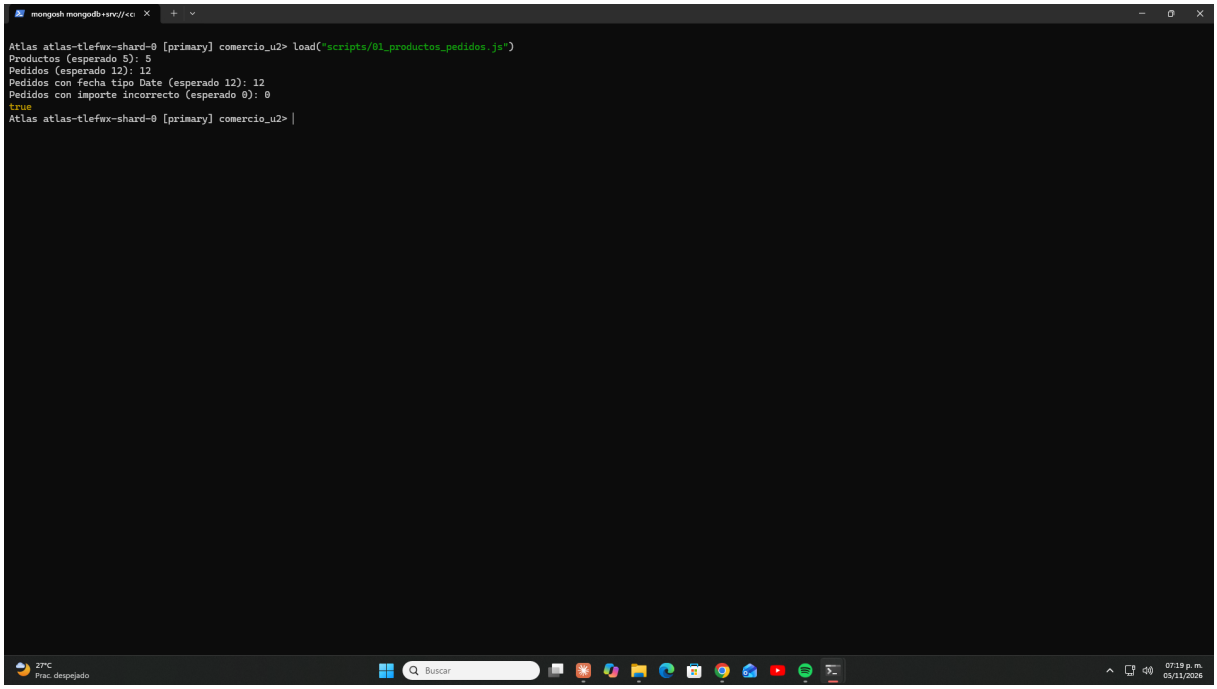
Si algo falla

Lo que sale	Qué pasa	Qué hago
Se queda cargando / timeout	Mi IP no está autorizada o tengo VPN.	Revisar la IP Access List.
Authentication failed	Usuario o contraseña incorrectos.	Volver a copiar la contraseña de la nota o generar una nueva en Atlas.
mongosh no se reconoce	PowerShell no encuentra el programa.	Cerrar y abrir PowerShell.

Paso 6. Cargar productos y pedidos (script 01)

Este script hace tres cosas: inserta los **5 productos**, crea la colección **pedidos** con un **validador $jsonSchema** y carga los **12 pedidos**. Se ejecuta dentro de mongosh:

```
cls
load("scripts/01_productos_pedidos.js")
```



Lo que debe salir: Productos 5, Pedidos 12, fechas tipo Date 12 e importes incorrectos 0. El **`true`** final solo significa que el archivo se cargó bien.

Lo más importante que hay dentro del script

```
db.productos.insertMany([ { sku: "P01", nombre: "Computadora portatil", ... } ])
db.createCollection("pedidos", { validator: { $jsonSchema: esquemaPedidos },
  validationLevel: "strict", validationAction: "error" })
db.pedidos.insertMany([ { _id: 1, clienteId: "C01", estado: "pendiente", ... } ])
```

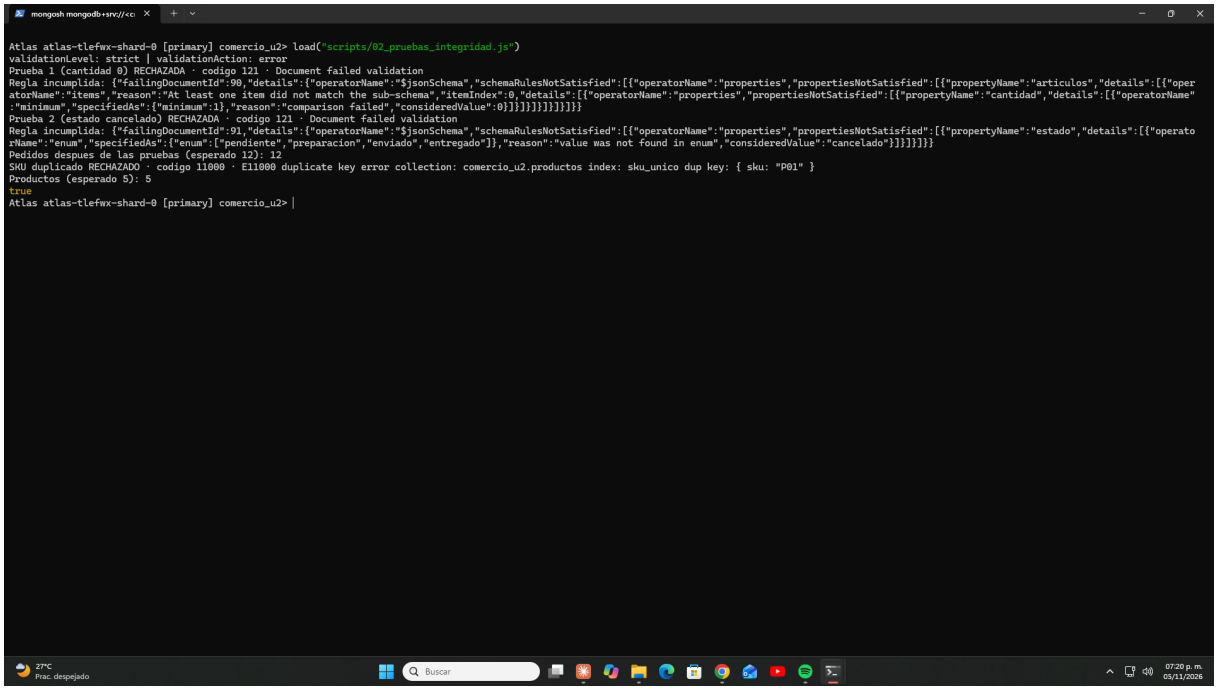
¿Qué significa? el **validador** es como un guardia en la puerta: revisa cada pedido antes de dejarlo entrar. Pide que tenga todos los campos obligatorios, que el estado sea uno de los permitidos (pendiente, preparacion, enviado, entregado), que la cantidad sea mínimo 1 y que la fecha sea de tipo fecha. **strict** quiere decir que revisa inserciones y actualizaciones; **error** quiere decir que si algo no cumple, lo rechaza.

Decidí **incrustar** los artículos dentro de cada pedido (un arreglo «articulos» con sku, cantidad y precio) porque un pedido siempre se consulta completo. Los productos quedaron en su propia colección porque se comparten entre muchos pedidos y sus existencias cambian solas.

Paso 7. Probar que las reglas funcionan (script 02)

Aquí intento meter datos **incorrectos a propósito**. Si MongoDB los rechaza, es buena señal: la base se protege sola.

```
cls
load("scripts/02_pruebas_integridad.js")
```



Prueba	Qué intenté	Resultado
1	Un pedido con cantidad 0	Rechazado, código 121
2	Un pedido con estado «cancelado»	Rechazado, código 121
3	Un producto con SKU P01 repetido	Rechazado, código 11000

¿Qué significa? el **código 121** significa «Document failed validation»: el documento no cumplió las reglas del validador. En el texto largo (errInfo) se lee exactamente qué falló: **`"propertyName":"cantidad"`** con **`"minimum":1`**, o **`"consideredValue":"cancelado"`** que no está en la lista permitida. El **código 11000** significa «duplicate key»: ya existe un producto con ese SKU y el índice único **`sku_unico`** no deja repetirlo.

El índice único se creó con este comando:

```
db.productos.createIndex({ sku: 1 }, { unique: true, name: "sku_unico" })
```

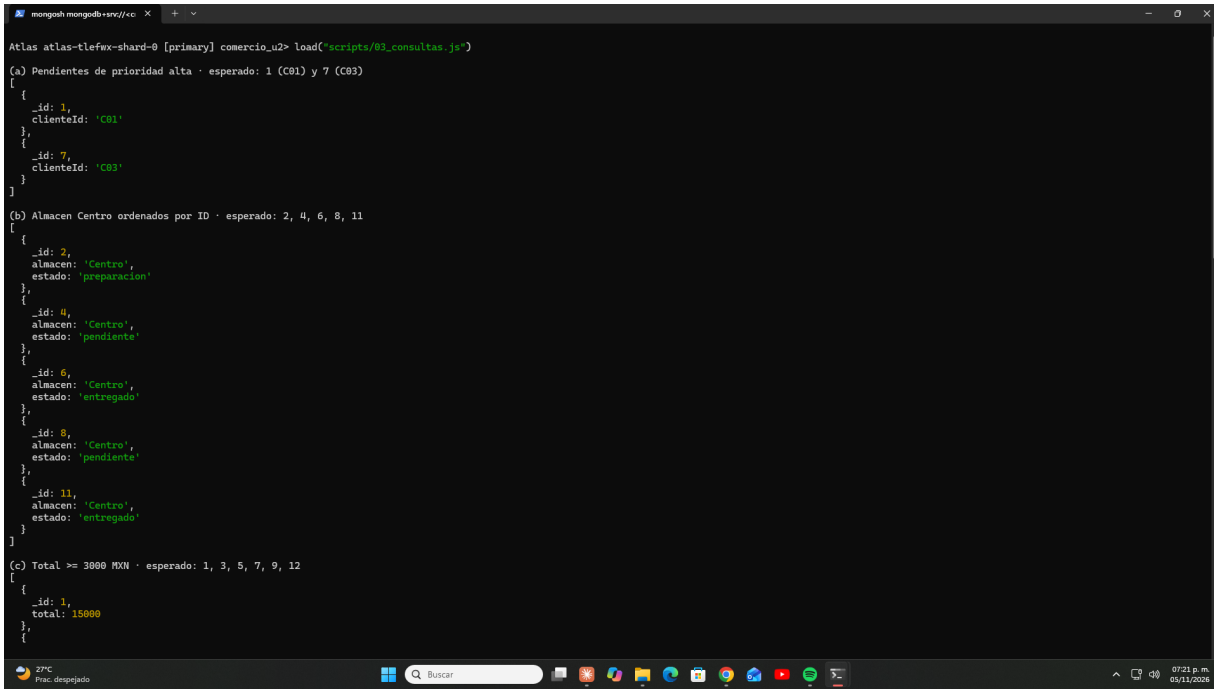
Cómo lo explico: *«Después de los rechazos siguen siendo 12 pedidos y 5 productos: ningún dato malo entró. El 121 es del validador y el 11000 del índice único.»*

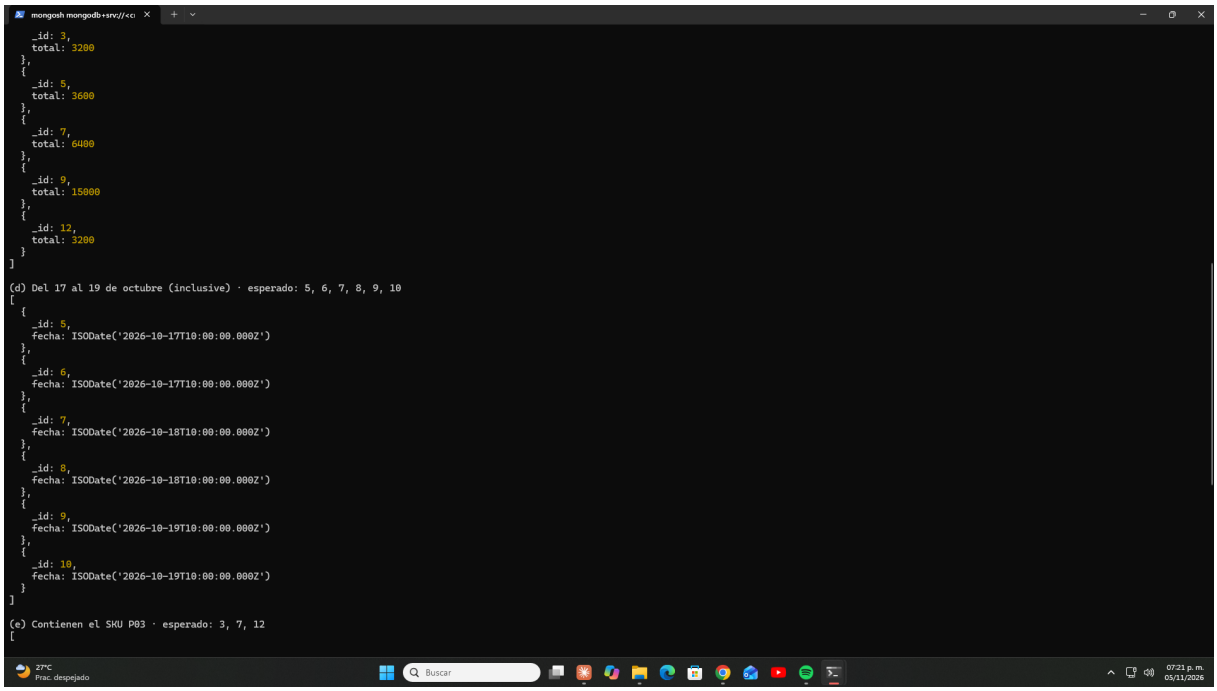
Paso 8. Hacer consultas (script 03)

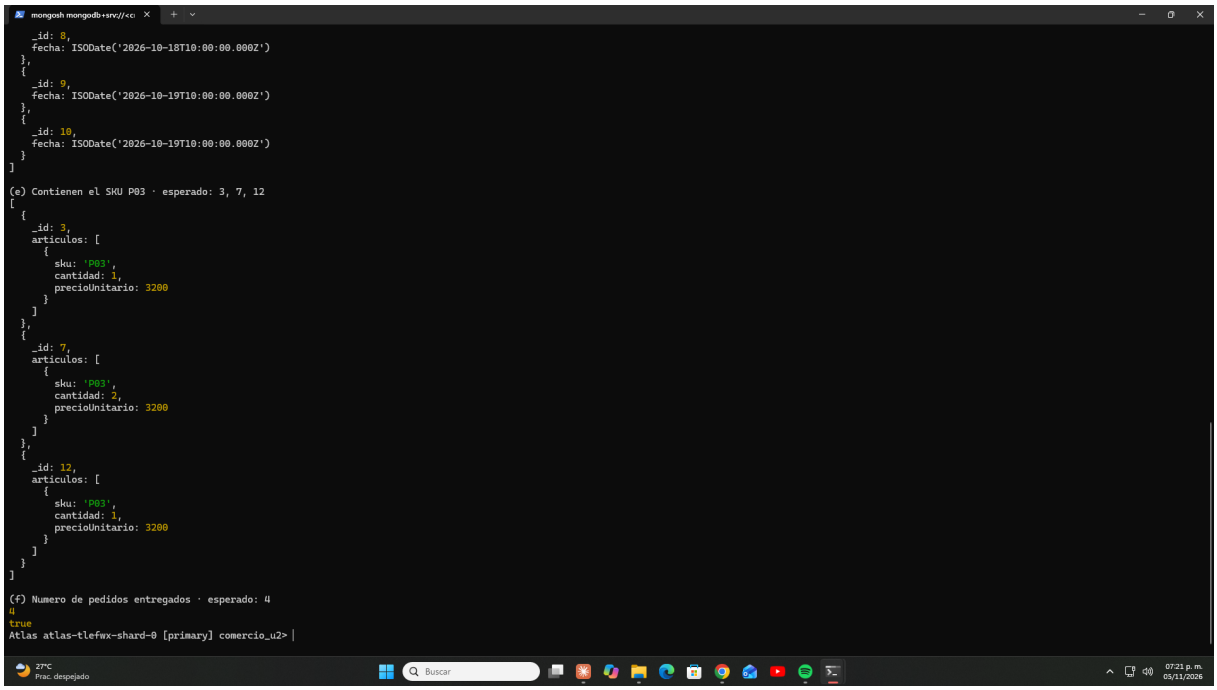
Ahora le hago seis preguntas a la base, como lo haría un gerente de la tienda. Antes de cada respuesta el script imprime lo que debería salir, para compararlo.

```
cls
load("scripts/03_consultas.js")
```

	Pregunta	Comando principal	Resultado
a	Pendientes de prioridad alta	`find({estado:"pendiente", prioridad:"alta"}, {_id:1, clienteId:1})`	1 y 7
b	Pedidos del almacén Centro, en orden	`find({almacen:"Centro"}).sort({_id:1})`	2, 4, 6, 8, 11
c	Total de 3000 o más	`find({total:{$gte:3000}})`	1, 3, 5, 7, 9, 12
d	Del 17 al 19 de octubre	`find({fecha:{$gte:...17, $lt:...20}})`	5 al 10
e	Pedidos con el monitor P03	`find({"articulos.sku":"P03"})`	3, 7, 12
f	Cuántos están entregados	`countDocuments({estado:"entregado"})`	4





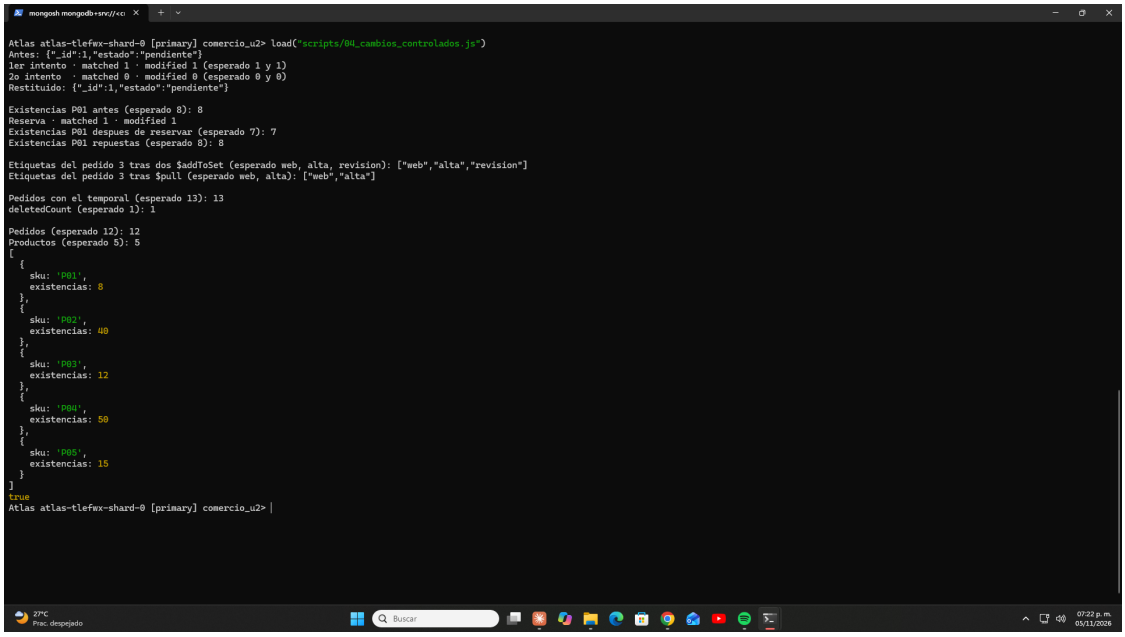


¿Qué significa? el segundo paréntesis de **`find`** es la **proyección**: elige qué campos mostrar (1 = mostrar). **`$gte`** es «mayor o igual» y **`$lt`** es «menor que». En la consulta de fechas usé «menor que el día 20» para que entre completo el día 19, sin importar la hora. Con **`"articulos.sku"`** (notación de punto) MongoDB busca dentro del arreglo de artículos. Las fechas salen como **ISODate**, lo que confirma que se guardaron como fecha real y no como texto.

Paso 9. Cambios controlados (script 04)

En una tienda real no se cambian datos «a lo loco». Cada cambio lleva una **condición** para que solo ocurra si el documento está en el estado esperado. El script cambia cosas y al final deja todo como estaba.

```
cls
load("scripts/04_cambios_controlados.js")
```

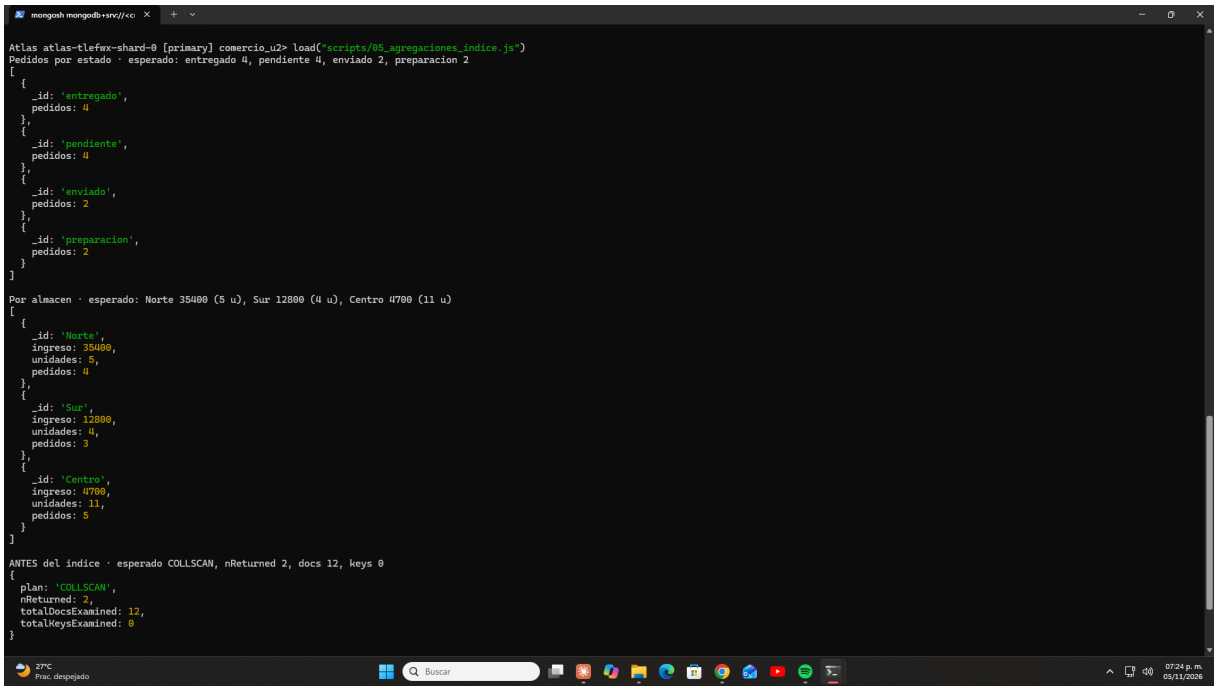


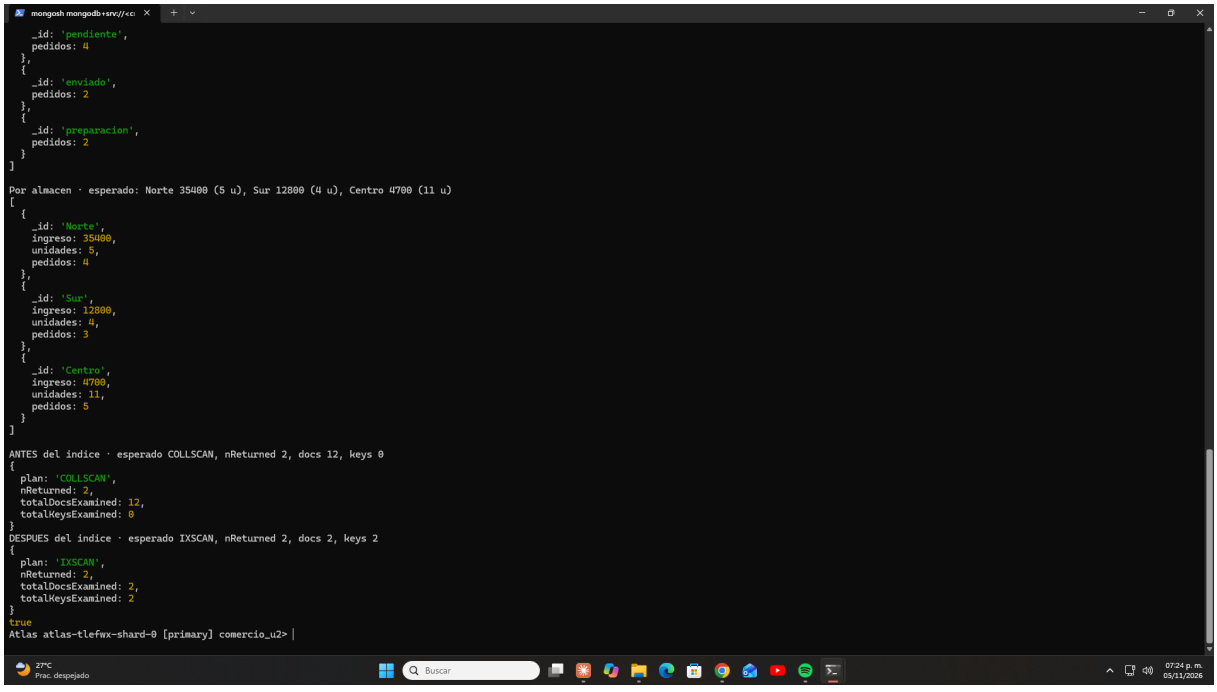
Qué hace	Comando principal	Resultado
Pasa el pedido 1 a preparación, solo si sigue pendiente	`updateOne({_id:1, estado:"pendiente"}, {$set:{estado:"preparacion"}})`	1.er intento 1 y 1; 2.º intento 0 y 0
Reserva una laptop solo si hay existencias	`updateOne({sku:"P01", existencias:{$gte:1}}, {$inc:{existencias:-1}})`	8 → 7 → 8
Agrega la etiqueta «revision» dos veces	`$addToSet: {etiquetas:"revision"}`	Solo se guarda una vez
Quita la etiqueta	`$pull: {etiquetas:"revision"}`	Quedan web y alta
Crea y borra un pedido temporal	`deleteOne({_id:99, temporal:true})`	13 → 12 pedidos

¿Qué significa? matched es cuántos documentos cumplieron el filtro y **modified** cuántos se cambiaron de verdad. En el segundo intento sale 0 y 0 porque el pedido ya no estaba pendiente: la condición lo protegió de un cambio repetido. **`$inc`** suma o resta a un número. **`$addToSet`** agrega a una lista sin repetir. **`$pull`** quita de una lista. El borrado usa el **_id** y **temporal: true**, así no se borra otro pedido por error.

Paso 10. Reportes y velocidad con índice (script 05)

```
cls
load("scripts/05_agregaciones_indice.js")
```





Los reportes (agregación)

Una **agregación** es como una línea de producción: los datos pasan por varias etapas y salen transformados.

- **`$group`** junta los pedidos por un campo (estado o almacén) y **`$sum`** va sumando.
- **`$unwind`** «desarma» la lista de artículos para poder sumar cada uno por separado.
- **`$addToSet`** con **`$size`** cuenta pedidos distintos sin repetir.
- **`$sort`** ordena el resultado; aquí de mayor a menor ingreso.

Almacén	Ingreso	Pedidos	Unidades
Norte	35 400	4	5
Sur	12 800	3	4
Centro	4 700	5	11

Por estado: entregado 4, pendiente 4, enviado 2 y preparación 2 (suman 12). Norte vende menos unidades pero gana más porque ahí están las laptops de 15 000.

El índice y cómo leer explain()

Un **índice** funciona como el índice de un libro: en lugar de leer todas las páginas, vas directo a la que buscas. Con **`explain("executionStats")`** MongoDB me dice cómo hizo la búsqueda.

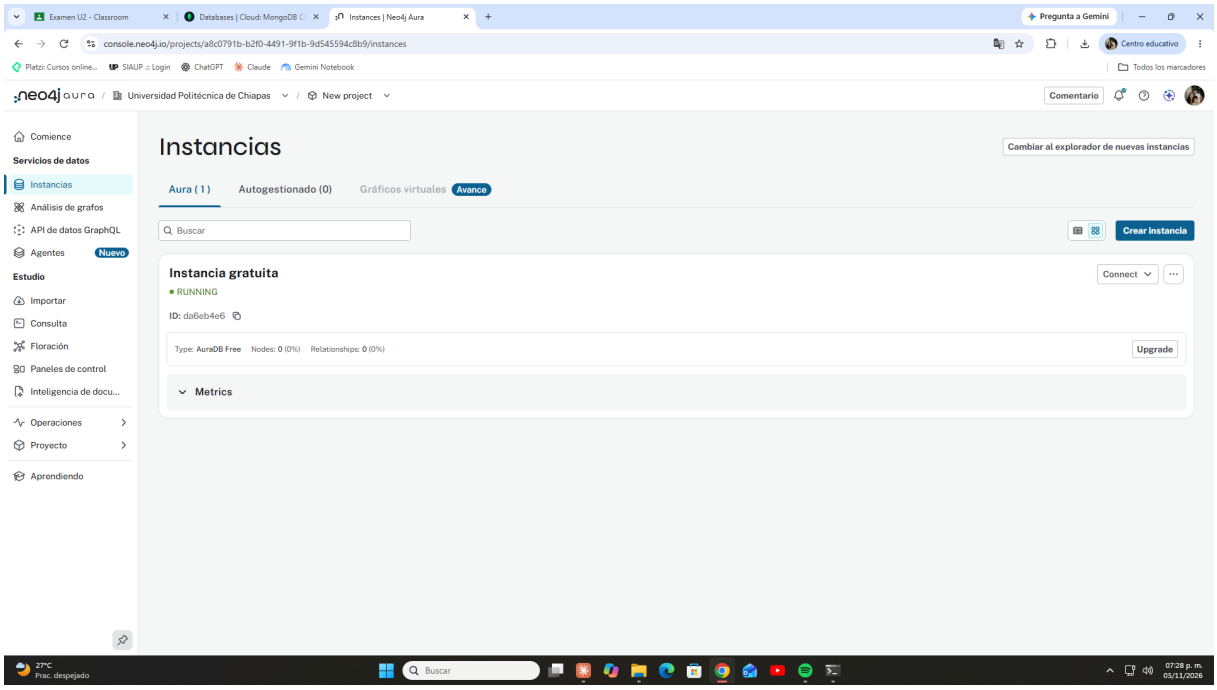
```
db.pedidos.find({estado:"pendiente", prioridad:"alta"}).explain("executionStats")
db.pedidos.createIndex({ estado: 1, prioridad: 1 })
```

Dato	Antes del índice	Después	Qué significa
plan	COLLSCAN	IXSCAN	COLLSCAN = revisó toda la colección. IXSCAN = usó el índice.
nReturned	2	2	Cuántos resultados devolvió. Es la misma respuesta.
totalDocsExamined	12	2	Cuántos documentos tuvo que leer. Bajó de 12 a 2.
totalKeysExamined	0	2	Cuántas entradas del índice revisó. Antes 0 porque no había índice.

Cómo lo explico: *«Con 12 pedidos la diferencia es pequeña, pero con un millón de pedidos leer solo los que coinciden es muchísimo más rápido.»*

Paso 11. Crear la base de grafos en Neo4j AuraDB

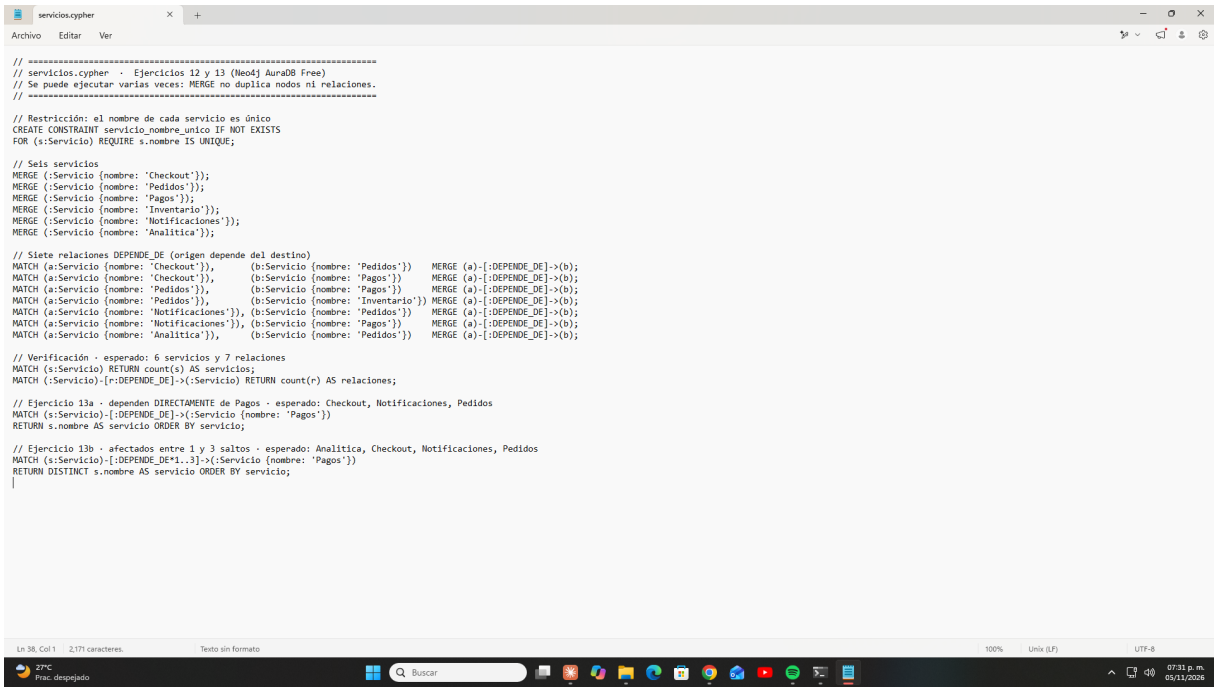
1. Entré a **`console.neo4j.io`** con mi cuenta de Google.
2. Clic en **Create instance** y elegí **AuraDB Free**.
3. Neo4j muestra el usuario **neo4j** y una contraseña. Di clic en **Download and continue**: se descarga un .txt con la contraseña. Lo dejé en Descargas, nunca en la carpeta de la entrega.
4. Esperé 2 a 5 minutos hasta que dijera **RUNNING**.



¿Qué significa? RUNNING = encendida. **Type: AuraDB Free** = plan gratuito. **Nodes 0 / Relationships 0** = todavía vacía.

Paso 12. Cargar el grafo de servicios

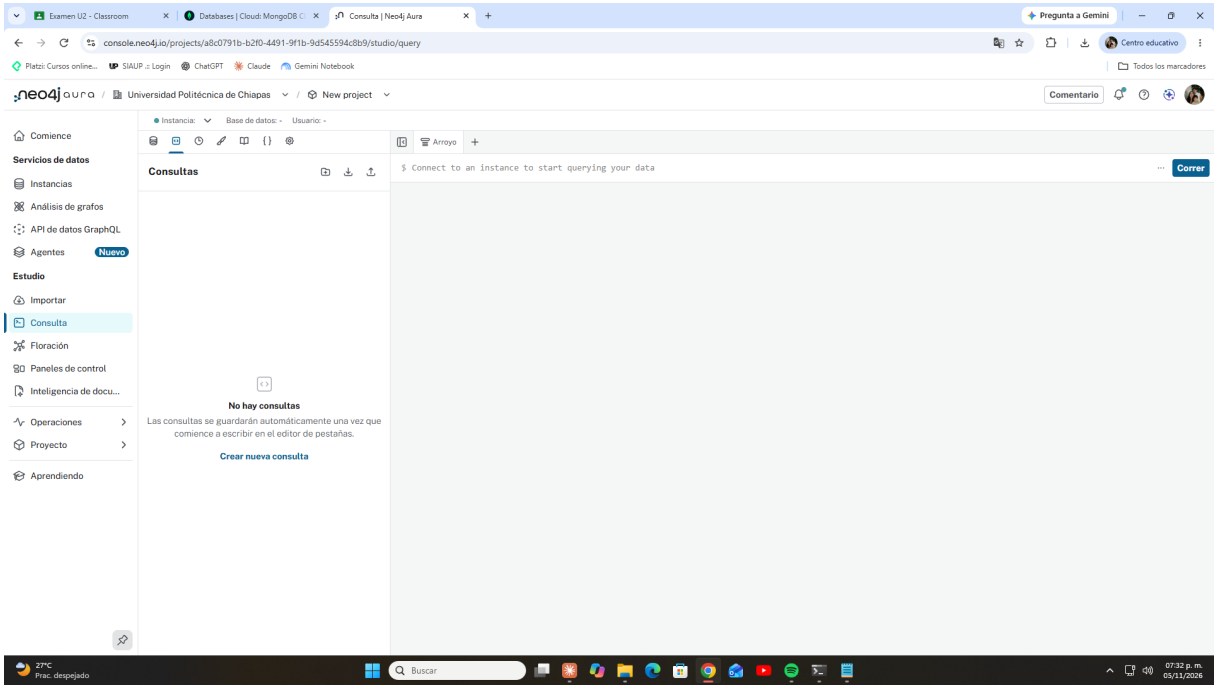
Abrí el archivo **`servicios.cypher`** con el Bloc de notas (clic derecho → Abrir con → Bloc de notas), seleccioné todo con **`Ctrl + A`** y lo copié con **`Ctrl + C`**.

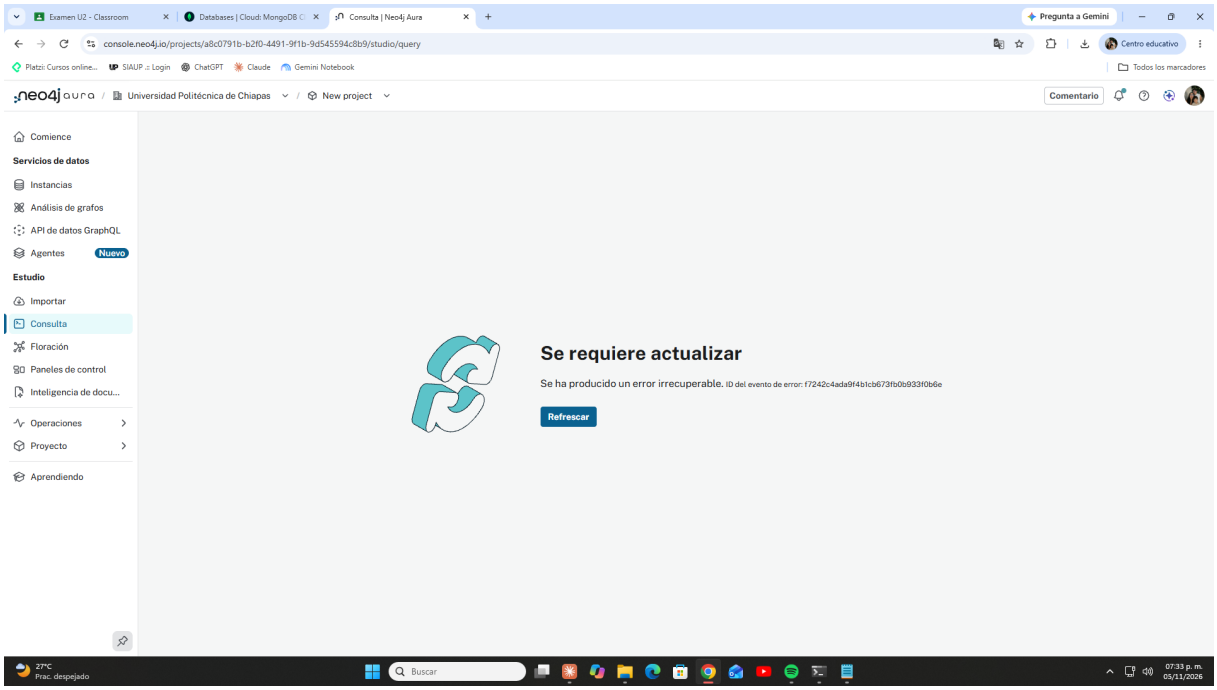


¿Qué significa? **`CREATE CONSTRAINT`** pone una regla: no puede haber dos servicios con el mismo nombre. **`MERGE`** significa «créalo solo si no existe»; así, si ejecuto el archivo dos veces, no se duplica nada. **`(a)-[:DEPENDE_DE]->(b)`** se lee «a depende de b»; la flecha marca la dirección.

Un problema que me salió: el traductor de Chrome

Al abrir **Query**, la página estaba traducida al español («Correr», «Arroyo») y de repente salió «Se requiere actualizar. Se ha producido un error irrecuperable».

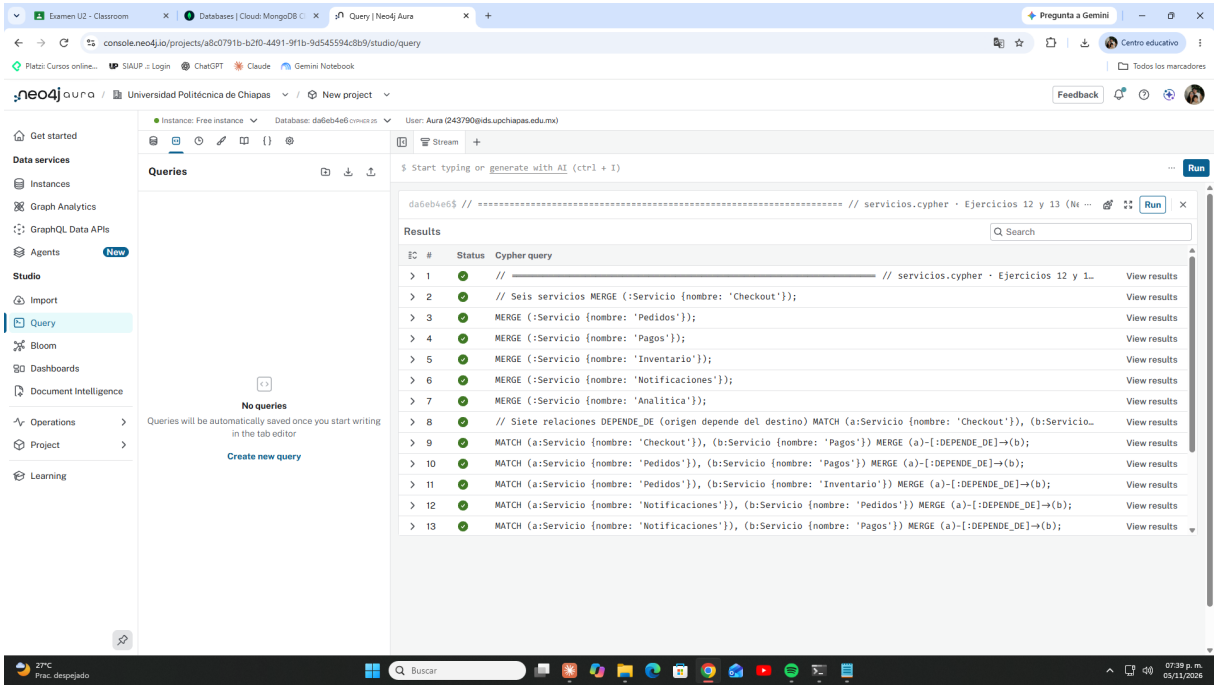


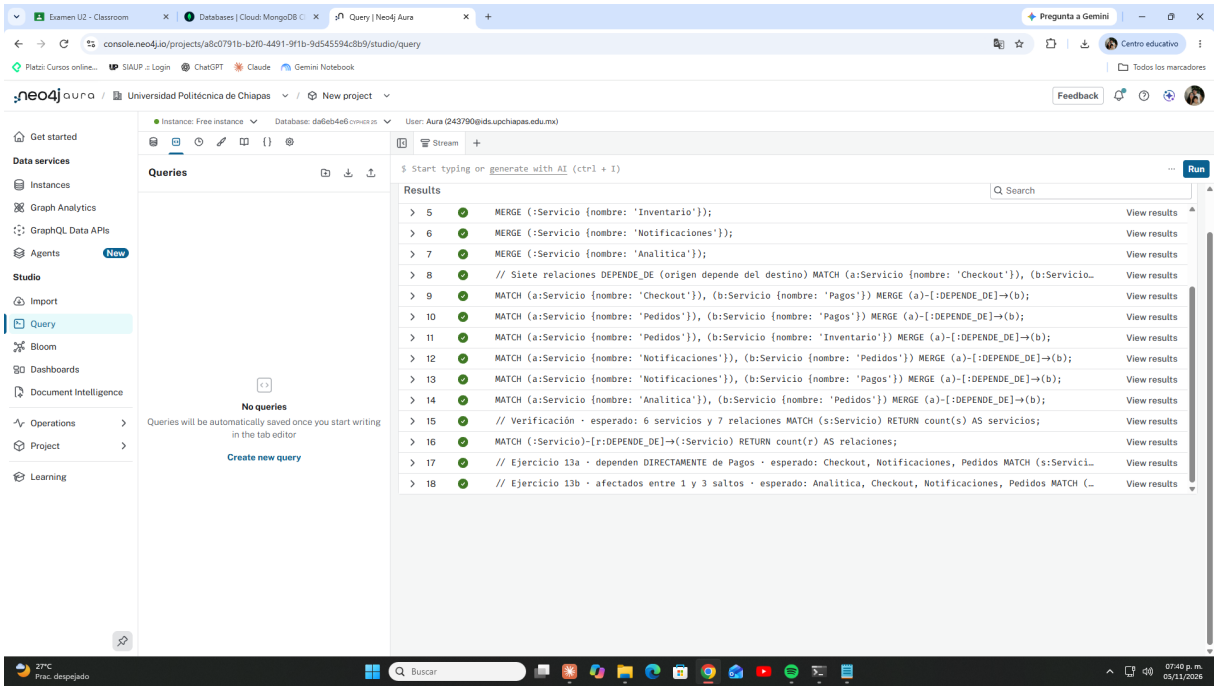


La causa era el traductor de Chrome, que modifica la página mientras funciona. La solución: clic en el ícono de traducir junto a la estrella, elegir **Mostrar original**, marcar **No traducir nunca este sitio** y refrescar con **F5**.

Ejecutar el script

1. Arriba, en **Instance**, elegí mi instancia y me conecté con usuario **neo4j** y su contraseña.
2. Pegué el script con **`Ctrl + V`** en el recuadro y di clic en **Run**.

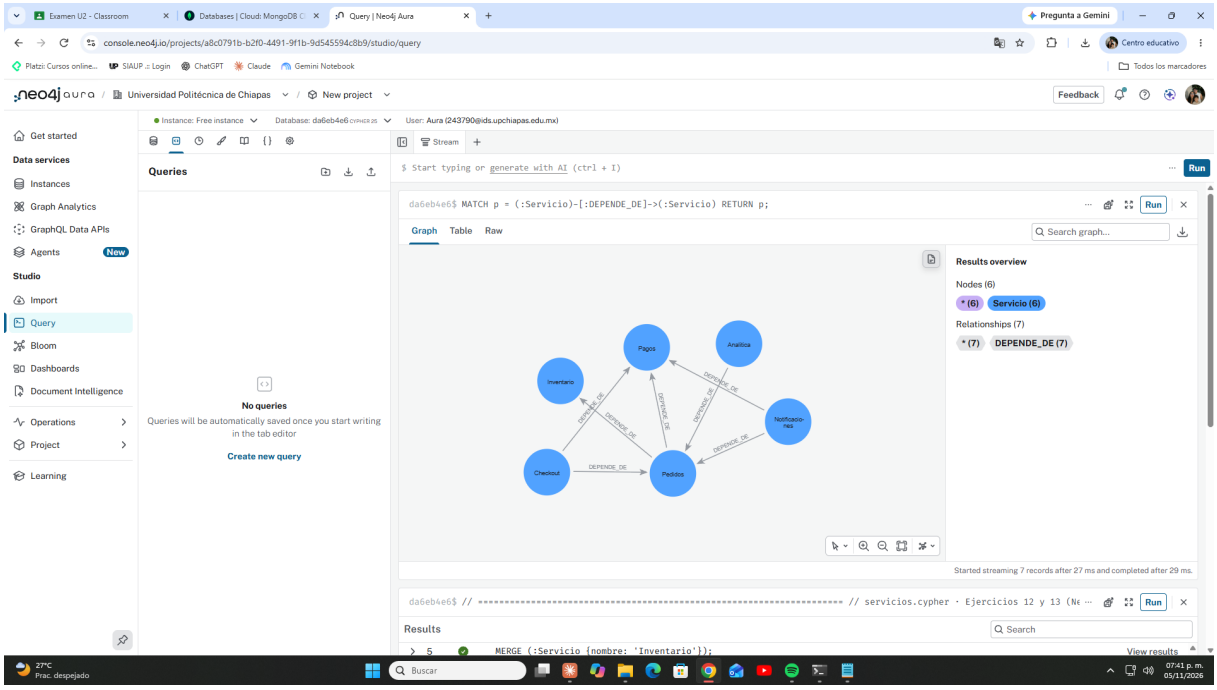




¿Qué significa? las **18 instrucciones** tienen palomita verde: la restricción, los 6 servicios, las 7 relaciones y las consultas. Algunas filas muestran el comentario pegado al código; es solo cómo Neo4j lo muestra en una línea.

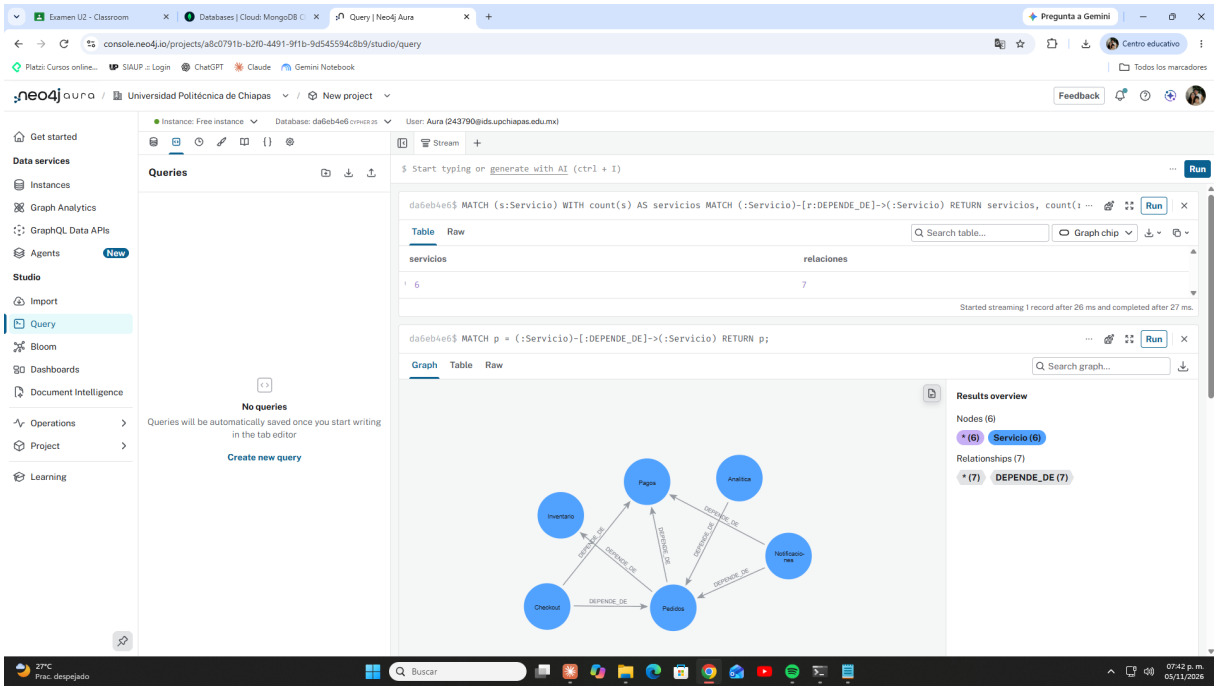
Paso 13. Ver el grafo y contarlo

```
MATCH p = (:Servicio)-[:DEPENDE_DE]->(:Servicio)
RETURN p;
```



¿Qué significa? **`MATCH`** busca un patrón: «un servicio que depende de otro servicio». Cada **círculo** es un nodo :Servicio y cada **flecha** es una relación DEPENDE_DE. En **Results overview** se confirma: Nodes (6) y Relationships (7). Se nota que **Pagos** y **Pedidos** reciben muchas flechas: muchos servicios dependen de ellos, así que son los más críticos.

```
MATCH (s:Servicio)
WITH count(s) AS servicios
MATCH (:Servicio)-[r:DEPENDE_DE]->(:Servicio)
RETURN servicios, count(r) AS relaciones;
```

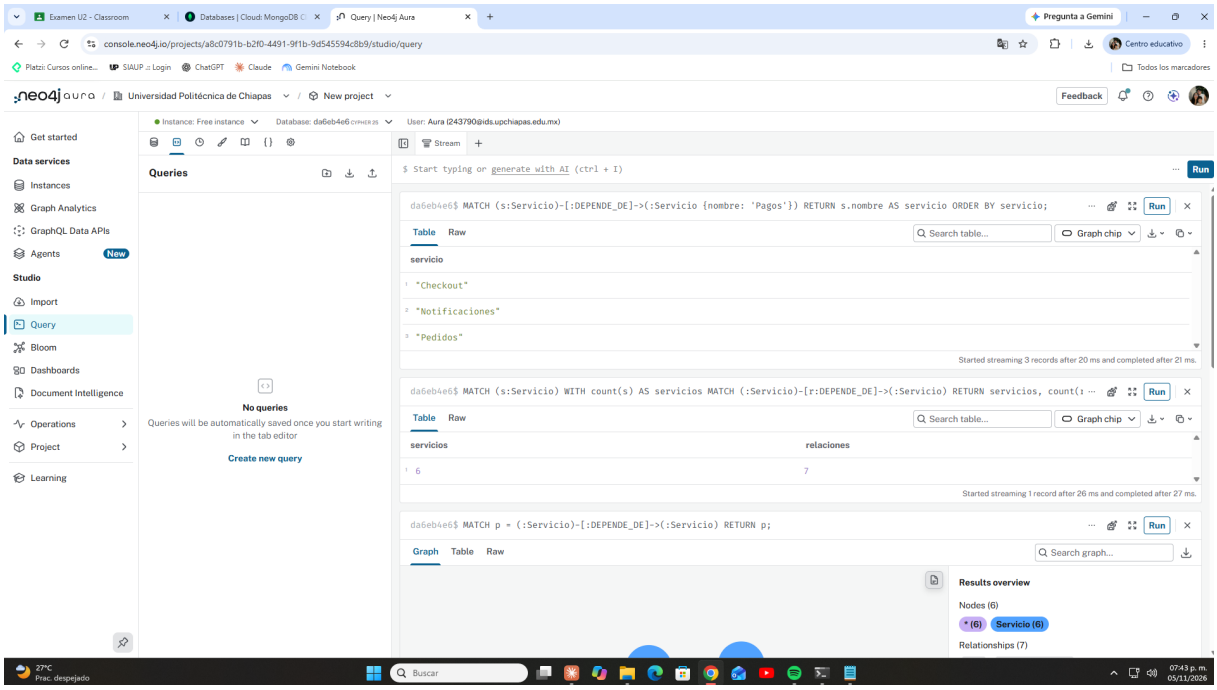


Lo que debe salir: servicios 6 y relaciones 7.

Paso 14. ¿Qué se afecta si falla Pagos?

Dependencia directa (una sola flecha)

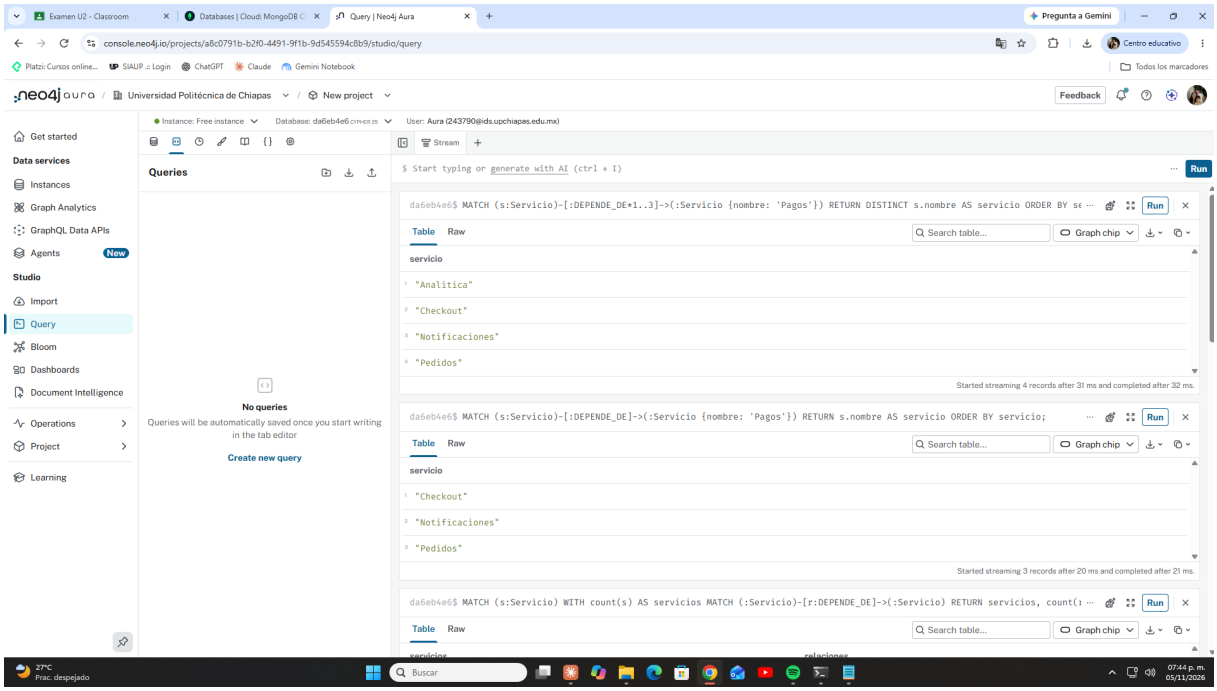
```
MATCH (s:Servicio)-[:DEPENDE_DE]->(:Servicio {nombre: 'Pagos'})
RETURN s.nombre AS servicio ORDER BY servicio;
```



Lo que debe salir: Checkout, Notificaciones y Pedidos.

Dependencia en cadena (de 1 a 3 saltos)

```
MATCH (s:Servicio)-[:DEPENDE_DE*1..3]->(:Servicio {nombre: 'Pagos'})
RETURN DISTINCT s.nombre AS servicio ORDER BY servicio;
```



Lo que debe salir: Analitica, Checkout, Notificaciones y Pedidos.

¿Qué significa? `*1..3` significa «sigue las flechas de 1 hasta 3 pasos». **Analítica** no depende de Pagos directamente, pero depende de Pedidos, y Pedidos depende de Pagos; por eso solo aparece en la segunda consulta. **`DISTINCT`** evita repetidos: Checkout llega a Pagos por dos caminos (directo y a través de Pedidos) y sin DISTINCT saldría dos veces.

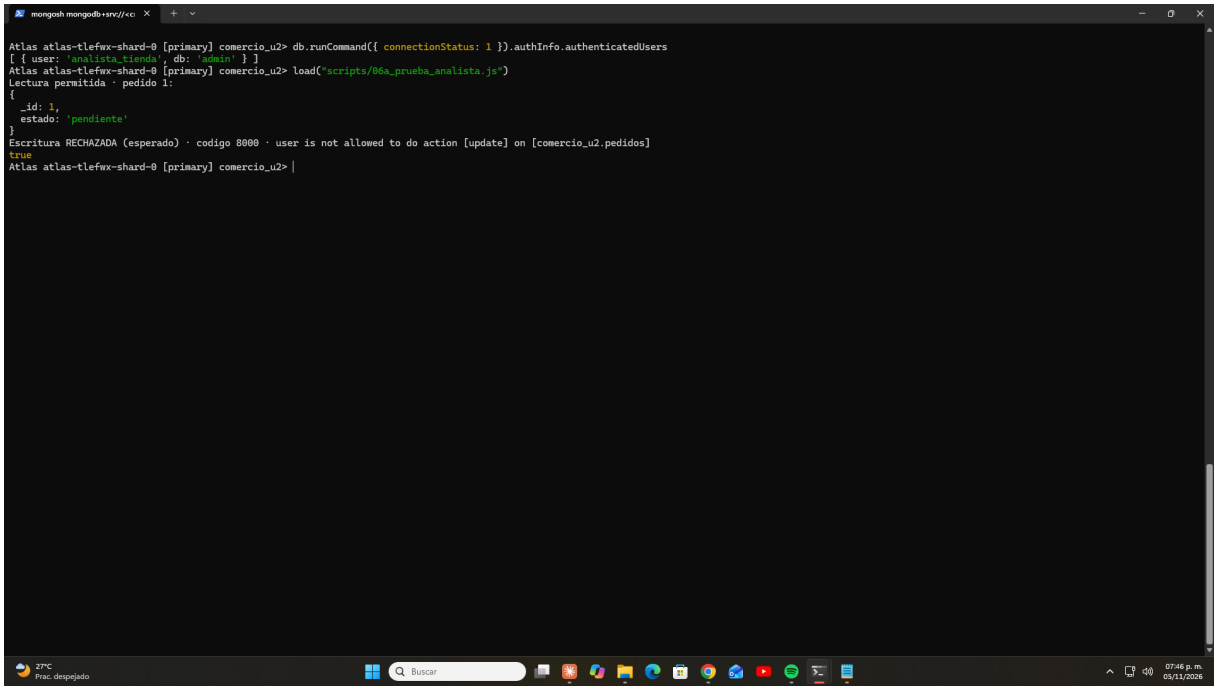
Cómo lo explico: *«Un grafo responde muy fácil preguntas de impacto en cadena, que en tablas necesitarían muchos JOIN.»*

Paso 15. Probar los permisos del analista

Ahora salgo de admin_tienda y entro como **analista_tienda** para comprobar que puede leer pero no modificar.

```
exit
mongosh "mongodb+srv://comercio-u2.zepcsbi.mongodb.net/comercio_u2" `
  --apiVersion 1 --username analista_tienda
```

```
cls
db.runCommand({ connectionStatus: 1 }).authInfo.authenticatedUsers
load("scripts/06a_prueba_analista.js")
```

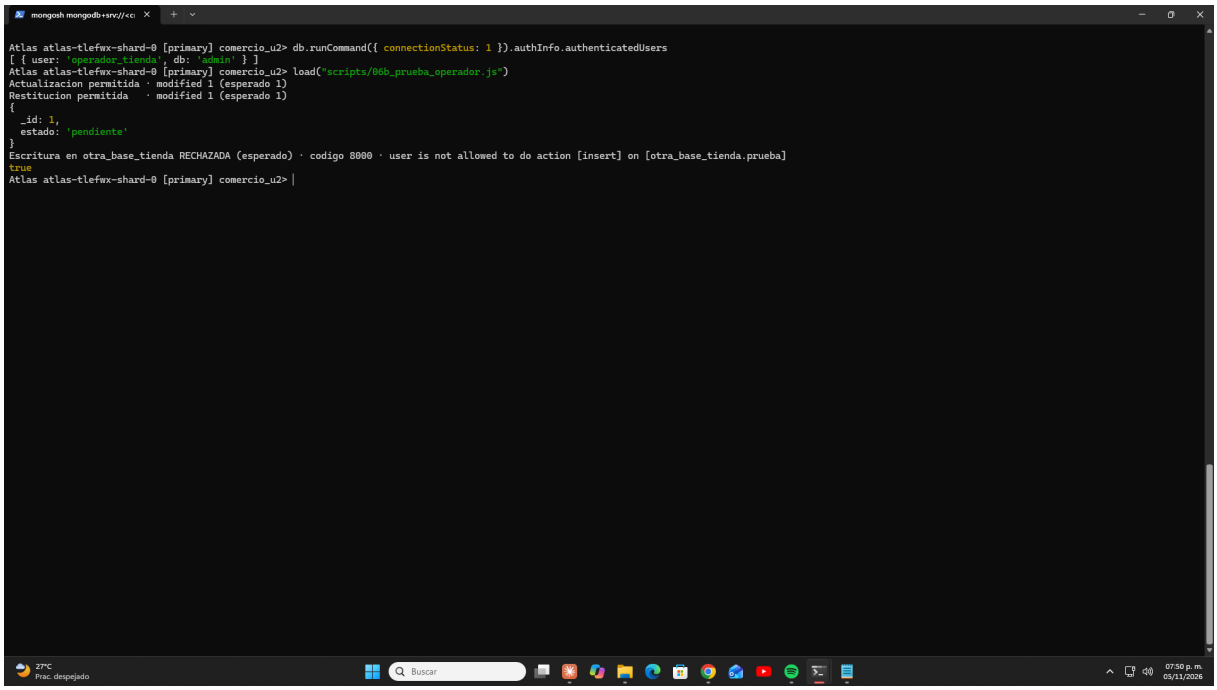


¿Qué significa? la primera línea confirma con qué usuario estoy conectado: **analista_tienda**. Pudo **leer** el pedido 1, pero al intentar actualizarlo salió **«user is not allowed to do action [update]»**: no tiene permiso. El código es **8000** porque en el plan gratuito de Atlas los permisos los revisa una capa de Atlas, que usa su propio código. En un MongoDB instalado en una computadora saldría el código 13 (Unauthorized); los dos significan lo mismo.

Paso 16. Probar los permisos del operador

```
exit
mongosh "mongodb+srv://comercio-u2.zepcsbi.mongodb.net/comercio_u2" `
  --apiVersion 1 --username operador_tienda

cls
db.runCommand({ connectionStatus: 1 }).authInfo.authenticatedUsers
load("scripts/06b_prueba_operador.js")
```



¿Qué significa? el operador **sí pudo** cambiar el pedido 1 y regresarlo a pendiente (modified 1 y 1), porque tiene readWrite en comercio_u2. Pero al intentar escribir en **otra_base_tienda** fue rechazado: su permiso no llega fuera de la base de la tienda.

Candado	Qué decide	Dónde se configura
Red	Desde dónde se puede conectar	IP Access List
Autenticación	Quién eres (usuario y contraseña)	Database Users
Autorización	Qué puedes hacer (roles)	Specific Privileges

Cómo lo explico: *«Tener la IP permitida no da permiso de escribir: son candados distintos. El analista entra desde una IP válida, pero su rol solo le deja leer.»*

Paso 17. Monitoreo: qué significan las gráficas

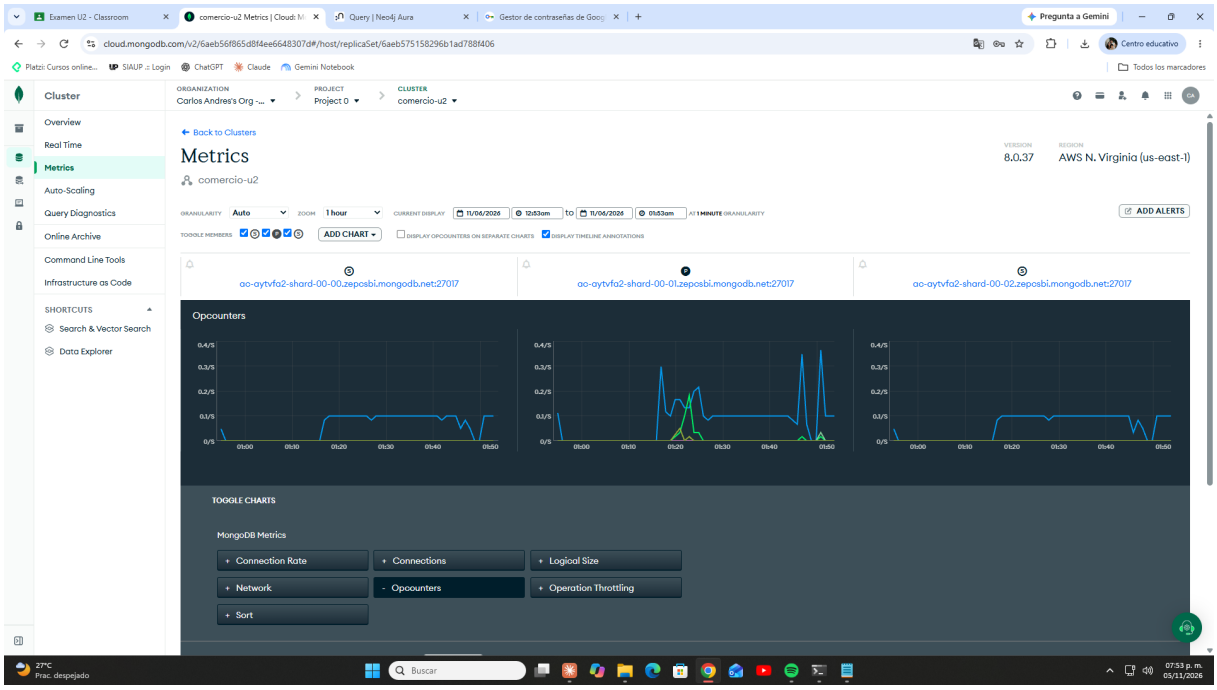
En Atlas: **Clusters** → comercio-u2 → **Metrics**. Arriba elegí **Zoom: 1 hour** para ver la última hora. Abajo, en **Toggle Charts**, se eligen las gráficas que quiero ver.

Lo primero: hay tres gráficas porque hay tres servidores

El clúster M0 es un **replica set**: tres servidores que guardan la misma información. Arriba de cada gráfica aparece su nombre (shard-00-00, shard-00-01 y shard-00-02) y una letra:

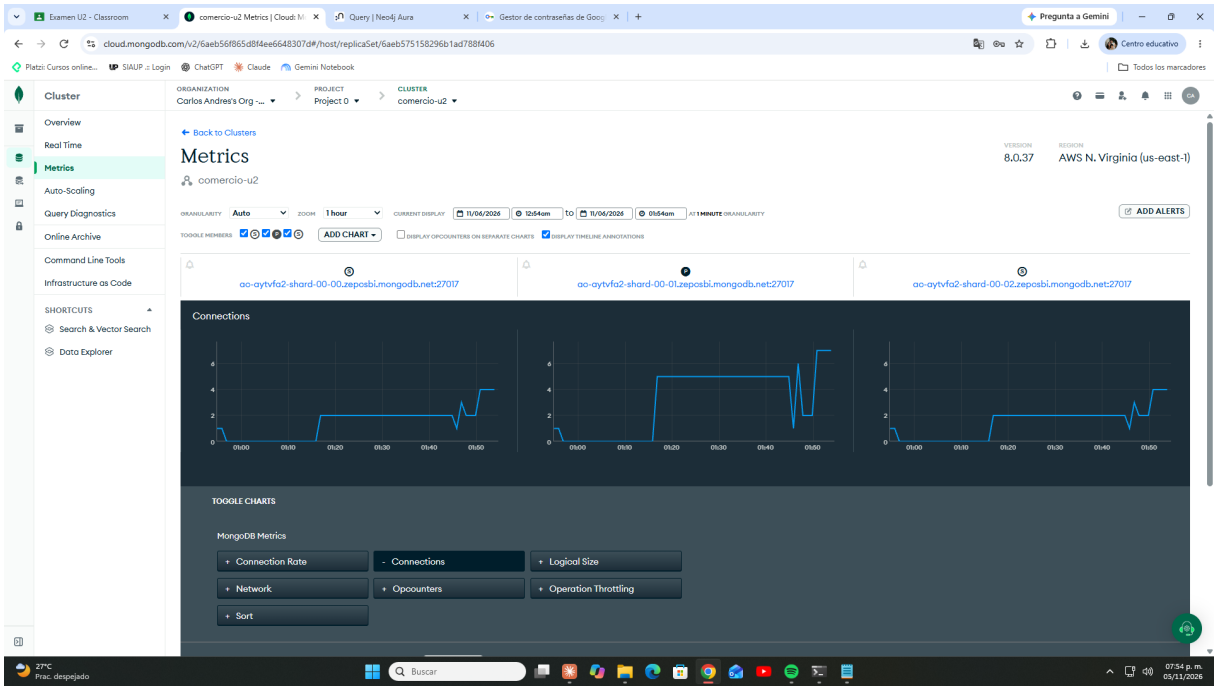
- **P (primario)**: el servidor principal. Recibe mis comandos, las escrituras y las lecturas.
- **S (secundario)**: copias de respaldo. Se sincronizan con el primario. Si el primario falla, uno de ellos toma su lugar automáticamente (a eso se le llama alta disponibilidad).

Gráfica Opcounters (operaciones por segundo)



Mide cuántas operaciones por segundo recibe cada servidor (consultas, inserciones, actualizaciones, borrados y comandos). Cada color es un tipo de operación. Las **subidas** en el primario (P, en medio) son los momentos en que ejecuté los scripts; los picos llegan a unos 0.35 operaciones por segundo. En los secundarios la línea es plana y baja (0.1/s): es el trabajo constante de copiar los datos del primario y comprobar que siguen conectados.

Gráfica Connections (conexiones abiertas)



Mide cuántos programas están conectados a cada servidor. Al principio está en 0. Cuando me conecté con mongosh, la línea subió; cuando escribí **`exit`** para cambiar de usuario, bajó un momento y volvió a subir con la nueva conexión. Al final había **7 conexiones en el primario** y **4 en cada secundario**.

¿Qué significa? no son 7 personas: Atlas también cuenta sus propias conexiones internas (monitoreo y sincronización entre servidores). El primario tiene más porque ahí llegan mis conexiones de mongosh. El límite del plan gratuito es 500, así que estoy muy lejos.

Cómo leer los ejes y la hora

Elemento	Qué significa
Eje horizontal	El tiempo. Muestra 01:00 a 01:50 porque Atlas usa hora UTC, que va 6 horas adelante de Chiapas (01:00 UTC = 7:00 p. m.).
Eje vertical	La cantidad: operaciones por segundo (0.1/s, 0.2/s…) o número de conexiones (2, 4, 6).
Granularity: 1 minute	Cada punto de la línea representa un minuto.
Zoom: 1 hour	La gráfica muestra la última hora.

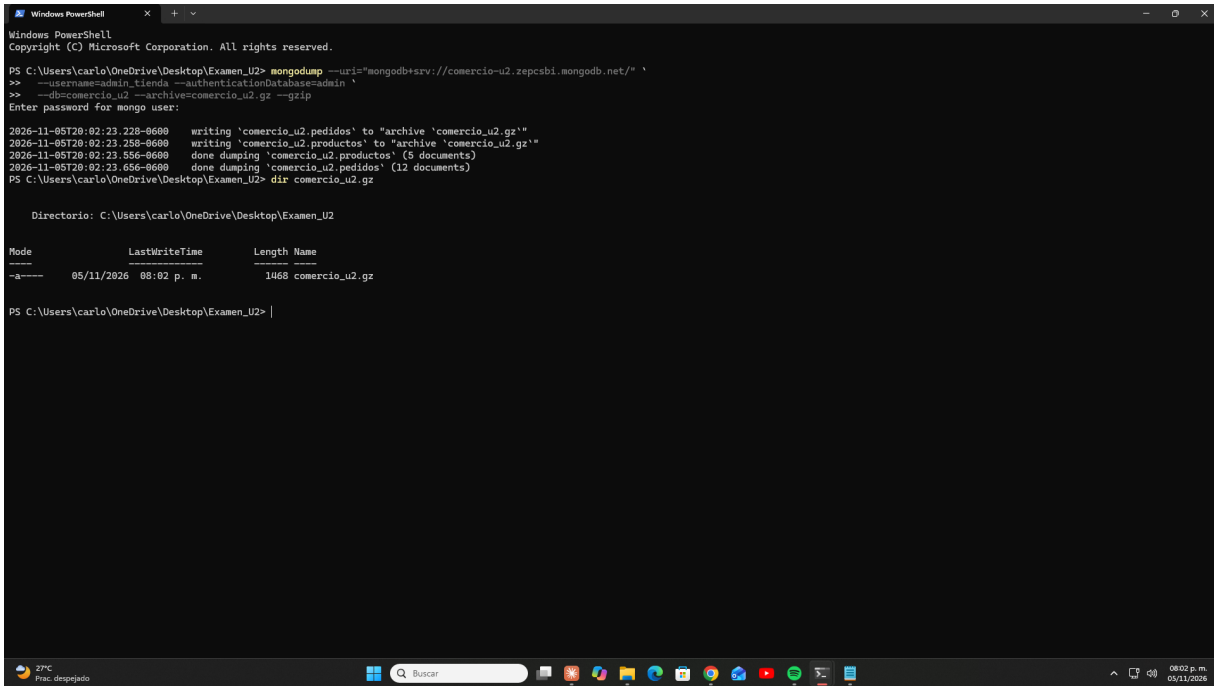
Cómo lo explico: *«El monitoreo sirve para detectar problemas: si las conexiones suben sin parar, alguna aplicación no está cerrando su sesión; si las operaciones se disparan, hay mucha carga.»*

Paso 18. Sacar un respaldo con mongodump

Ahora saco una copia de seguridad de toda la base comercio_u2 en un solo archivo comprimido. Esto se hace en **PowerShell**, no dentro de mongosh, así que primero salí con **`exit`**. Ojo: este comando **crea un archivo nuevo**, comercio_u2.gz, en la carpeta de trabajo.

```
mongodump --uri="mongodb+srv://comercio-u2.zepcsbi.mongodb.net/" `
  --username=admin_tienda --authenticationDatabase=admin `
  --db=comercio_u2 --archive=comercio_u2.gz --gzip

dir comercio_u2.gz
```



¿Qué significa? **`--archive`** guarda todo en un solo archivo y **`--gzip`** lo comprime; por eso pesa solo 1468 bytes. **`--authenticationDatabase=admin`** indica dónde vive el usuario en Atlas. Se respaldaron 5 productos y 12 pedidos.

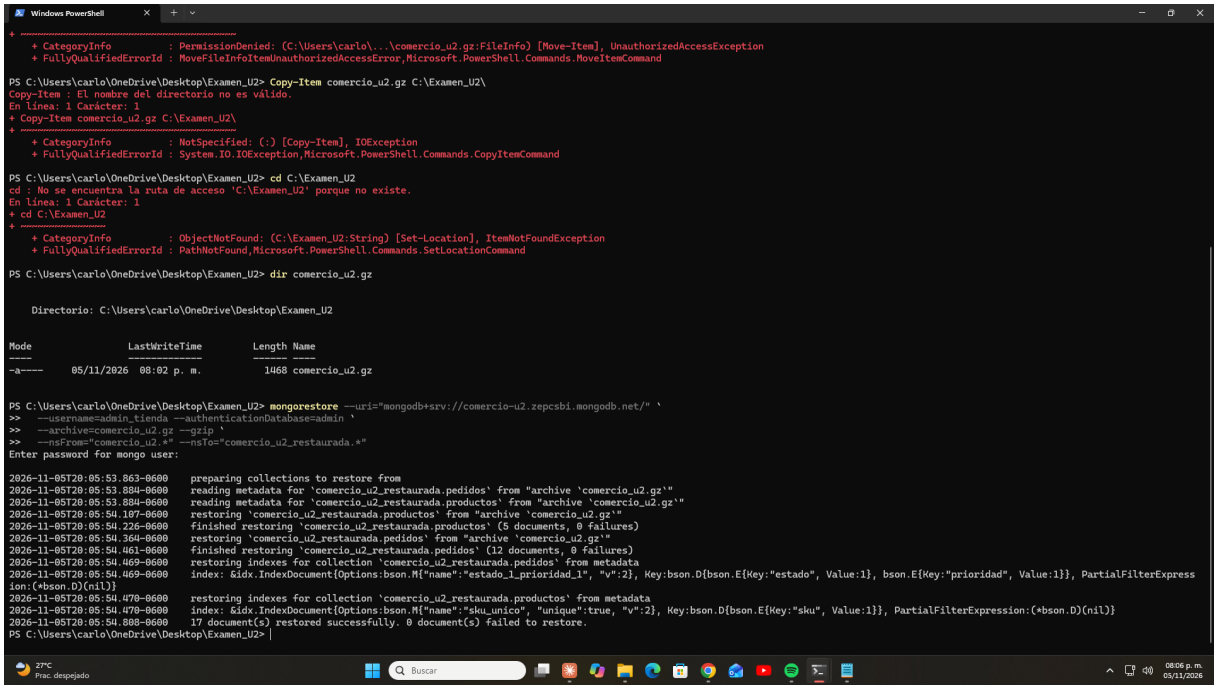
Un problema que me salió: no podía pegar la contraseña

Al copiar el comando, el portapapeles se reemplaza y la contraseña ya no está copiada. La solución fue: pegar el comando y dar Enter; cuando aparece **Enter password**, ir a la nota, copiar la contraseña, regresar y pegarla con clic derecho. Recordar que no se ve nada al pegarla.

Paso 19. Restaurar el respaldo en otra base

Un respaldo solo sirve si se puede recuperar. Lo restauré en una base **nueva y diferente** (comercio_u2_restaurada) para no tocar la original.

```
mongorestore --uri="mongodb+srv://comercio-u2.zepcsbi.mongodb.net/" `
  --username=admin_tienda --authenticationDatabase=admin `
  --archive=comercio_u2.gz --gzip `
  --nsFrom="comercio_u2.*" --nsTo="comercio_u2_restaurada.*"
```



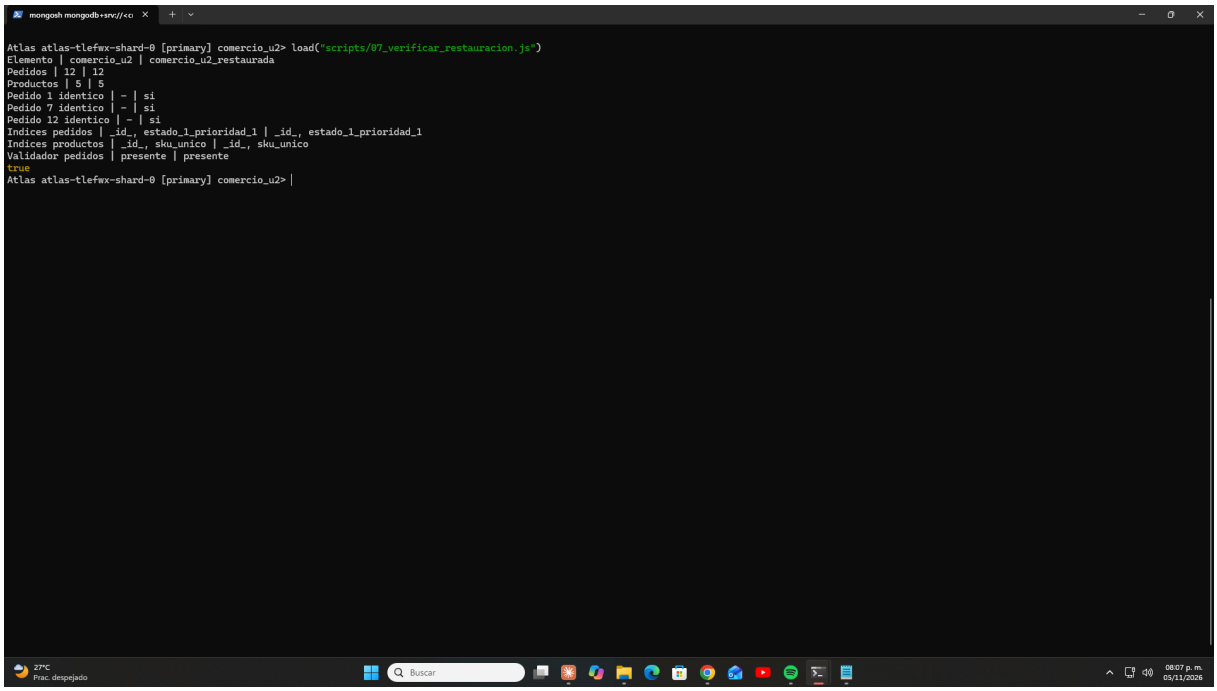
Las letras rojas de arriba son de un intento anterior de mover el archivo a otra carpeta que no existía (C:\Examen_U2); no afectaron nada. Lo importante es la parte de abajo.

¿Qué significa? --nsFrom y **--nsTo** cambian el nombre de la base al restaurar. Se restauraron los productos (5, 0 failures), los pedidos (12, 0 failures) y los índices **`estado_1_prioridad_1`** y **`sku_unico`**. La última línea dice **17 document(s) restored successfully. 0 document(s) failed**. No usé **`--drop`**, que borraría lo que ya existe.

Paso 20. Comprobar que la copia es idéntica

```
mongosh "mongodb+srv://comercio-u2.zepcsbi.mongodb.net/comercio_u2" `
  --apiVersion 1 --username admin_tienda

cls
load("scripts/07_verificar_restauracion.js")
```



¿Qué significa? el script pone lado a lado la base original y la restaurada: mismos 12 pedidos y 5 productos, los pedidos 1, 7 y 12 idénticos, los mismos índices y el validador presente en las dos. El respaldo es confiable.

Hay que saber dos límites del respaldo: guarda los datos **del momento** en que se hizo (lo que cambie después se perdería) y **no guarda la configuración de Atlas**: usuarios, contraseñas, roles ni la lista de IP.

Cómo lo explico: *«Un respaldo solo es confiable cuando se restaura y se comprueba. Lo restauré en una base distinta y verifiqué que fuera idéntico.»*